

N-body problem

A MCP meme collection
(and a project presentation)

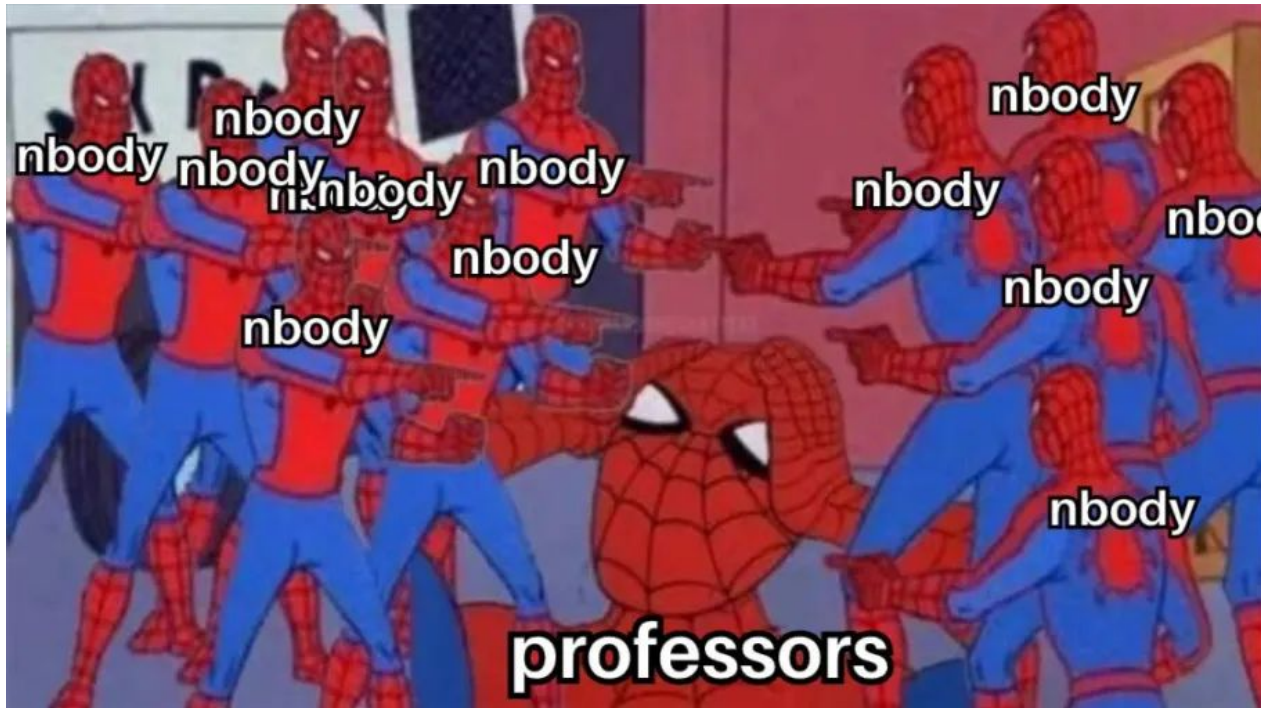
Brocco Luca



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

So it seems...

2



Project overview



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



The N-body problem

The N-body problem is a classical Computational Physics problems

It is known that, under standard conditions, the interaction between N bodies ($N > 2$) cannot be solved analytically

A numeric solution is however obtainable, and its accessibility is made easier by computers

We can simulate the evolution of system made by N bodies thanks to integration methods

We want to model pairwise gravitational forces:

$$F_{i,j} = G \frac{m_i m_j \hat{r}_{ij}}{r_{ij}^2}$$

the issue arises from theory when a third body is added: the system becomes chaotic, requiring an step-integration technique to provide a numerical solution

The net force acting on a particle is: $\sum_{i \neq j} \vec{F}_{i,j}$ sum of the gravitational forces from all other particles

For N particles, we are required to compute $\frac{N(N-1)}{2}$ interactions at each timestep

thus the computation scales as N^2 , requiring efficient coding choices

We have 2 updates available:

- 1) Euler method
simpler, but higher variance

$$x_{t+1} = x_t + v_t \delta t$$

$$v_{t+1} = v_t + a_t \delta t$$

- 2) Velocity verlet
more complex, but higher precision

$$x_{t+1} = x_t + v_t \delta t + \frac{1}{2} a_t \delta t^2$$

$$a_{t+1} = a_t + \frac{F_{t+1}}{m}$$

$$v_{t+1} = v_t + \frac{1}{2} [a_t + a_{t+1}] \delta t$$

We need to iterate one of these algorithm across a number of steps in order to compute the evolution of the system

The N-body problem

For easier Force computation we set $\mathbf{G} = 1$ and all quantities are set to have **arbitrary units**

We can convert from these to the SI:

consider a mass unit as **mass of the Earth**
and a length unit as **~ diameter of Earth**

$$M_T = 5.97 \cdot 10^{24} Kg$$

$$d_T = 1 \cdot 10^7 m$$

we then get the **time unit** with
where G is the standard constant
in SI units

$$T_0 = \sqrt{\frac{d^3}{GM_T}} = 1584s \sim 26min$$

Sequential solution



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

The sequential solution is the first implementation we propose
it is mostly used for:

- 1) ensuring correct interpretation of the problem in a simplified scenario
- 2) visualize trajectories for low N values
- 3) identifying parallelization / optimization opportunities

All our variables required to store floating point / involved in floating point operations will be **float**, while num_iter and N will be **int**

For this project I used the GPU available for the course: Jetson Nano 2GB

Running a sequential code leads all workload to land onto a single core:

```
Model: NVIDIA Jetson Nano 2GB Developer Kit - Jetpack 4.6.4 [L4T 32.7.4]
 1 [|||||99.0%] 1.5GHz 3 [|||||13.0%] 1.5GHz
 2 [||8.0%] 1.5GHz 4 [||6.1%] 1.5GHz
Mem [|||||1.6G/1.9G] FAN [0.0%] 0RPM
Swp [||90.6M/989M] Jetson Clocks: [running]
Emc [--- RUNNING ---] 1.6GHz 0% NV Power[0]: MAXN
Iram [0k/252k](lfb 252kB) Uptime: 0 days 0:28:13
GPU [0.0%] 921MHz
Dsk [#####25.0G/58.4G]

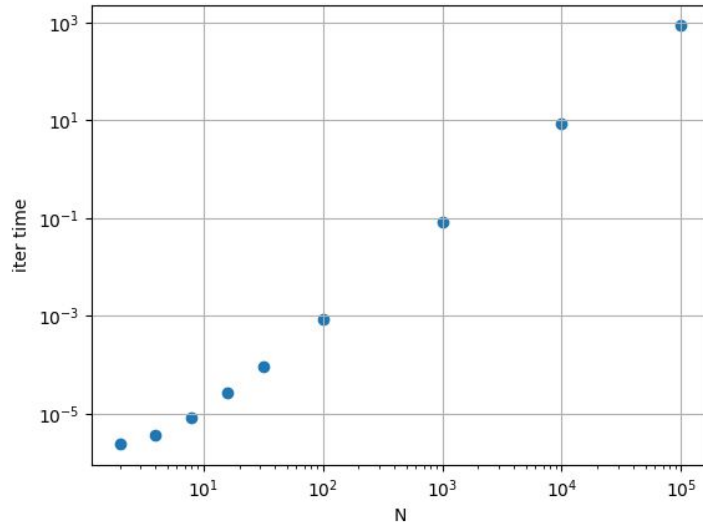
PID  USER  GPU  TYPE  PRI  S   CPU%  MEM  [GPU MEM]  Command
4284  root   I    G     20   S    0.0   2.2M  76k        nvargus-daemon
```

Main culprits of the code:

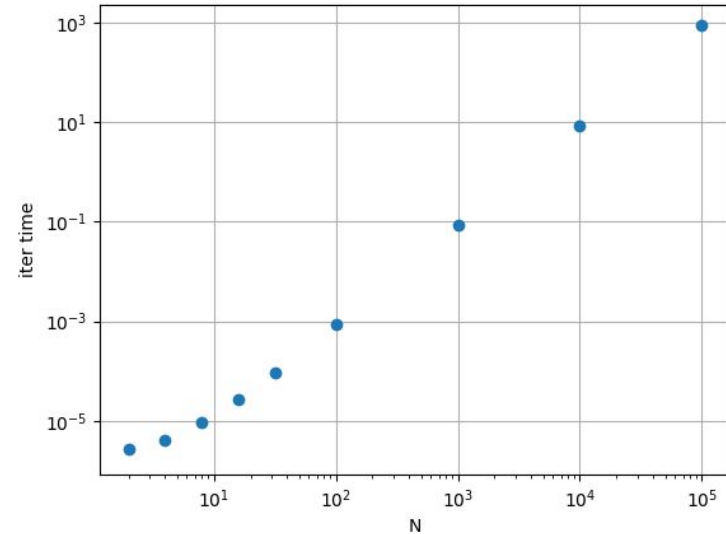
```
void compute_forces(int N, double epsilon, double x[][3], double m[], double F[][3], double *U_out) {  
    double U = 0.0, eps2 = epsilon*epsilon; // reset to zero and compute eps2  
    for (int i = 0; i < N; i++){  
        for (int k = 0; k < 3; k++){  
            F[i][k] = 0.0;  
        }  
    }  
    for (int i = 0; i < N; i++){  
        for (int j = 0; j < i; j++){  
            double dx = x[i][0]-x[j][0], dy = x[i][1]-x[j][1], dz = x[i][2]-x[j][2]; // distance  
            double r2 = dx*dx + dy*dy + dz*dz + eps2; // distance squared (faster than pow)  
            double inv_r = 1.0 / sqrt(r2);  
            double mimj = m[i] * m[j];  
            double coef = mimj * inv_r*inv_r*inv_r;  
            F[i][0]-=coef*dx; F[j][0]+=coef*dx; // exploit symmetry  
            F[i][1]-=coef*dy; F[j][1]+=coef*dy;  
            F[i][2]-=coef*dz; F[j][2]+=coef*dz;  
            U -= mimj * inv_r;  
        }  
    }  
    *U_out = U;  
}
```

Iteration time is expected to scale as N^2 ...

EULER



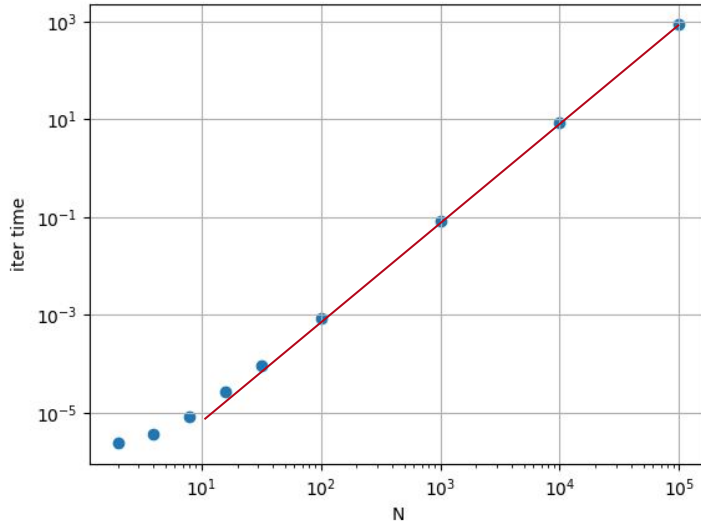
VERLET



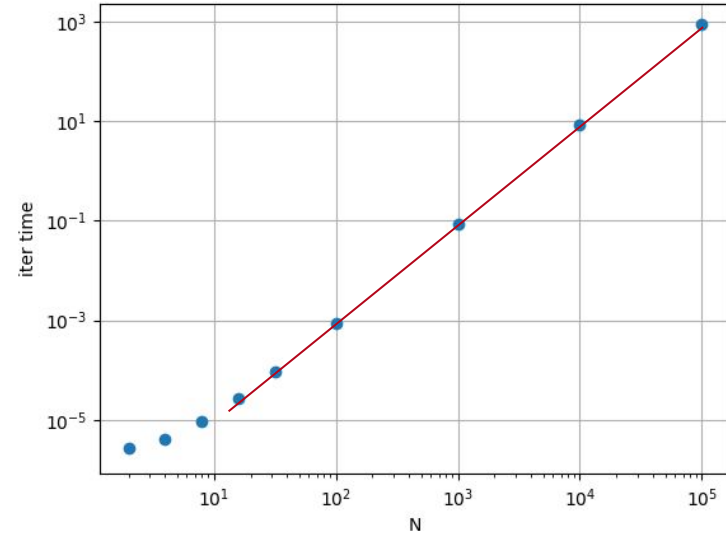
error bars are so **tiny** that they are not visible! Graphs are basically **overlapping**

Iteration time is expected to scale as N^2 ...

EULER



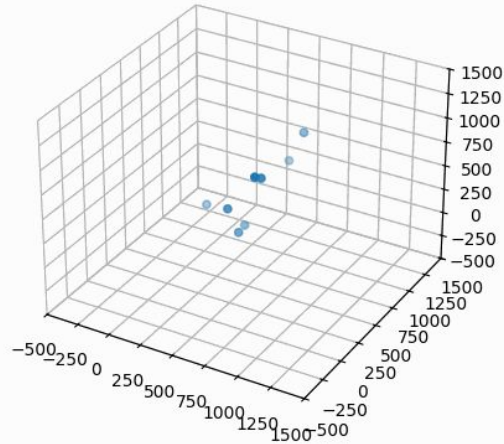
VERLET



linear in log-log with **m = 2**, as expected!

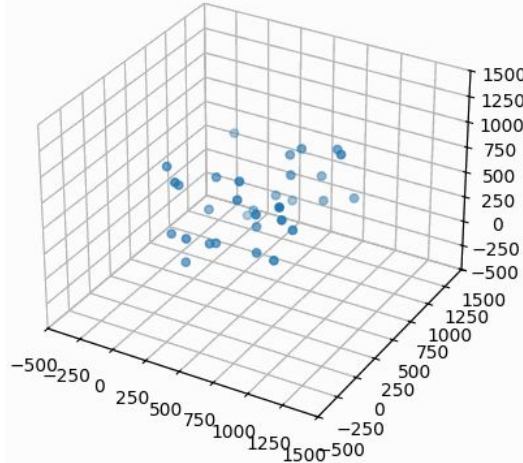
For small N we are able to plot and observe the evolution of the system

$t = 0$



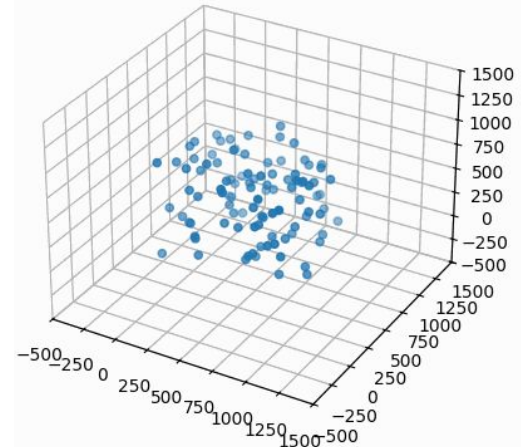
$N = 8$

$t = 0$



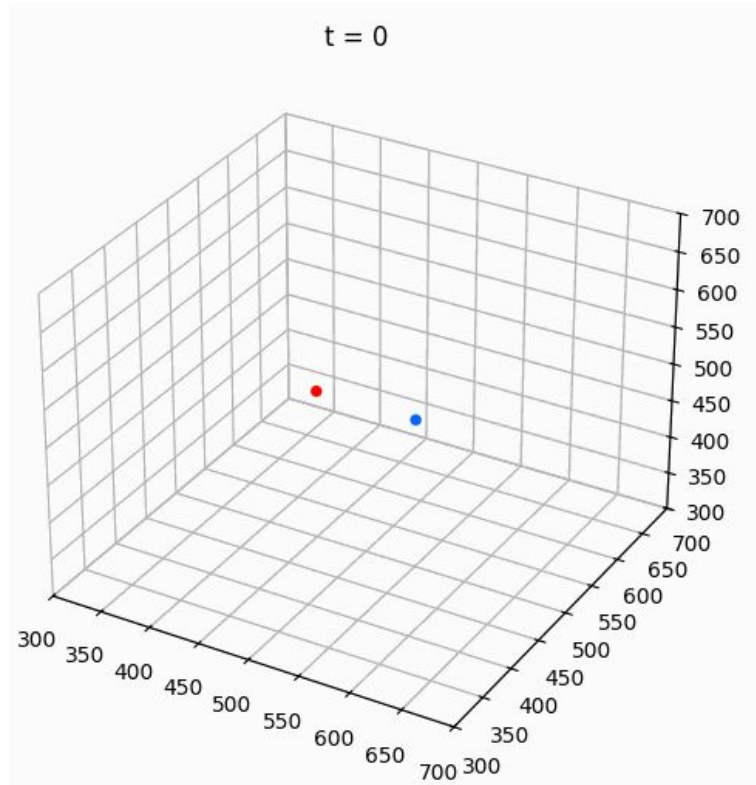
$N = 32$

$t = 0$

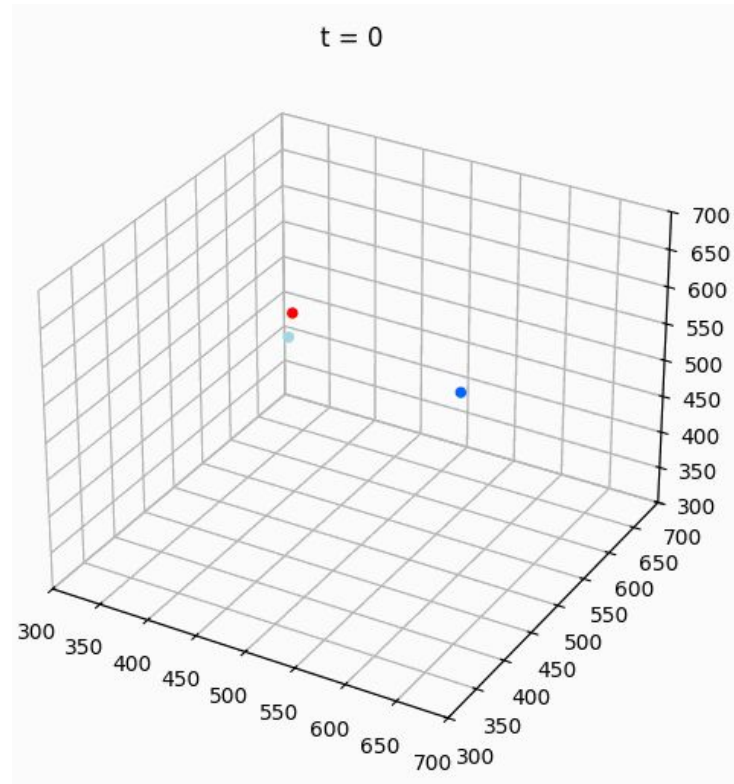


$N = 100$

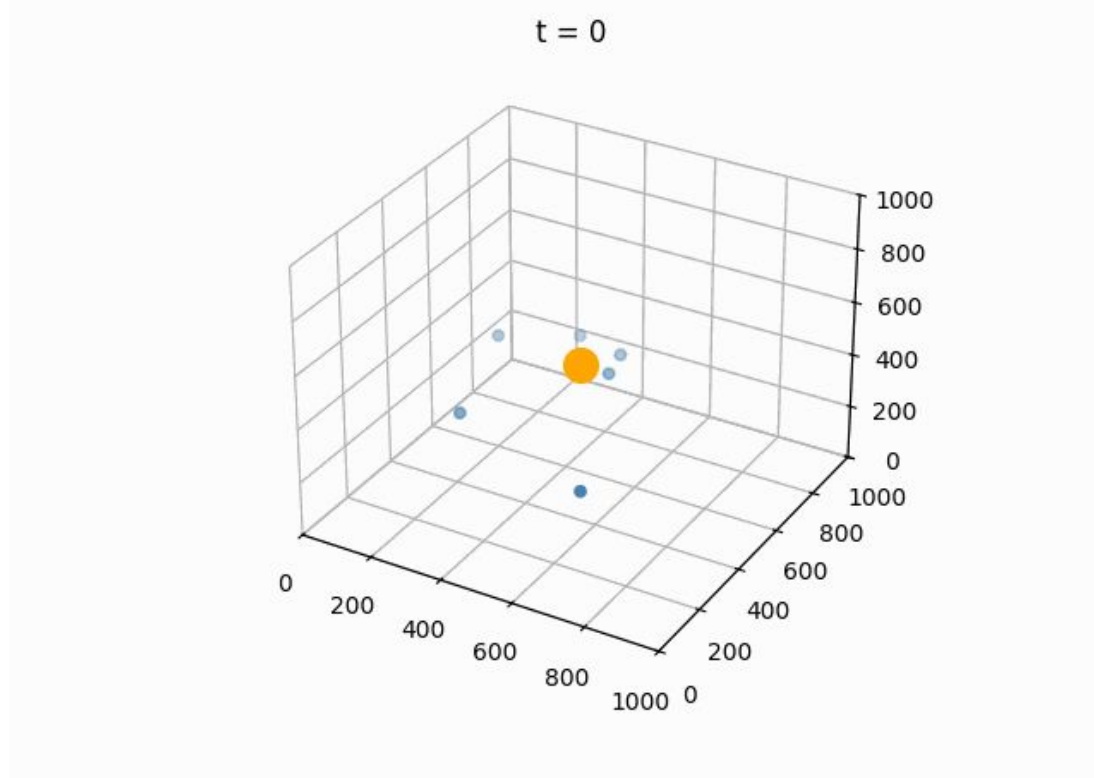
$N = 2$ allows us for visualization of elliptical trajectories (verlet)



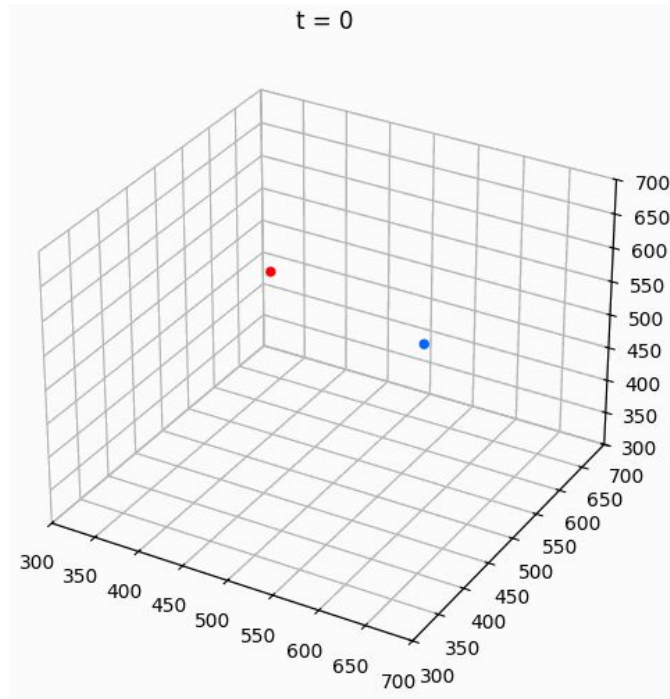
$N = 3$ is enough to have a chaotic system (verlet)



We can simulate a small star system and check its evolution (verlet)

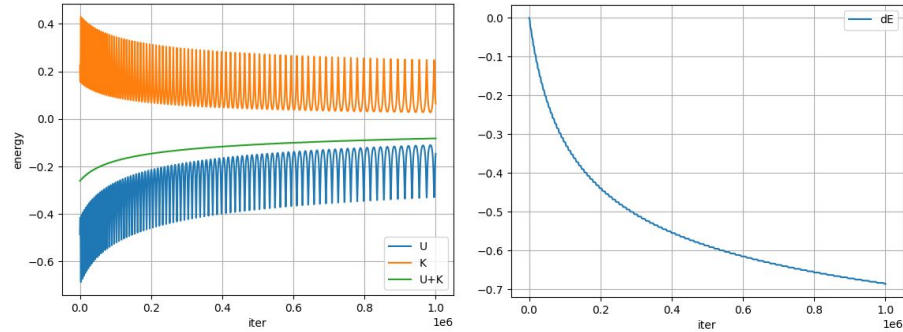


Interestingly enough, with $N = 2$ and Euler we observe an enlargement of trajectories:

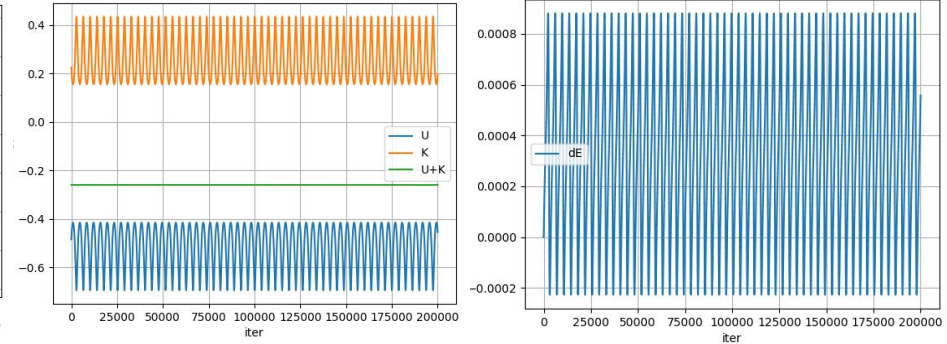


Phenomena observed in previous slide is related to **energy conservation**:

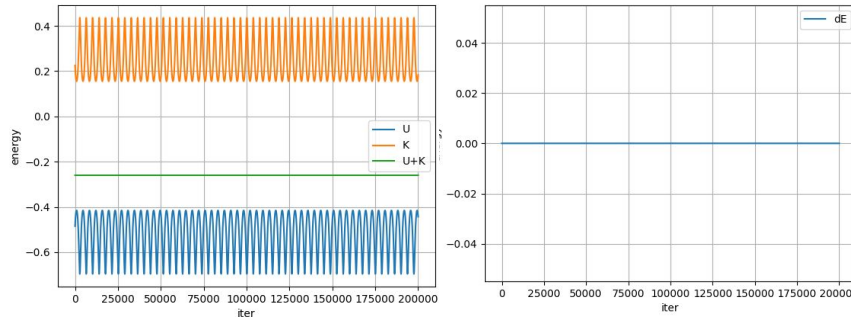
EULER



EULER-CROMER



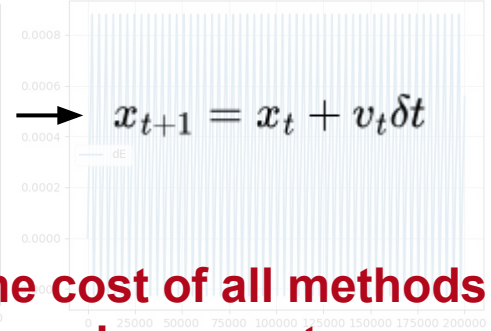
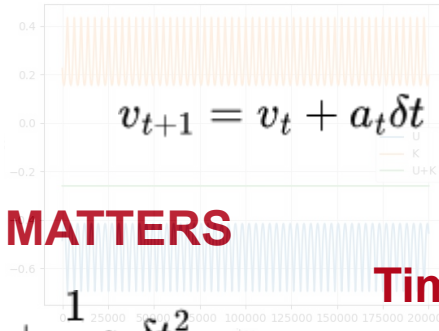
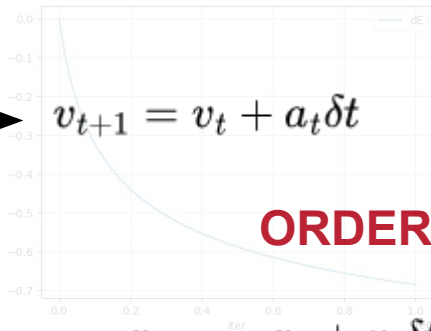
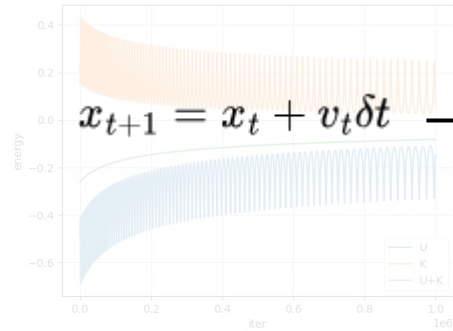
VERLET



Phenomena observed in previous slide is related to **energy conservation**:

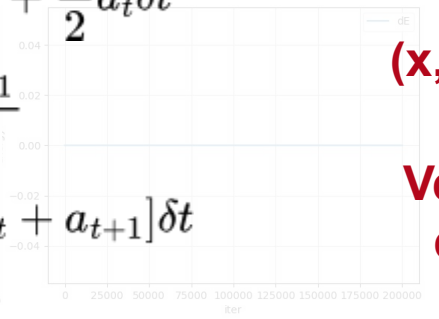
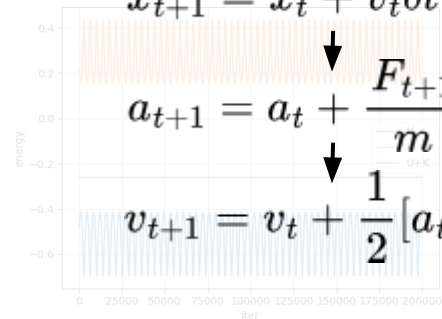
EULER

EULER-CROMER



ORDER MATTERS

VERLET



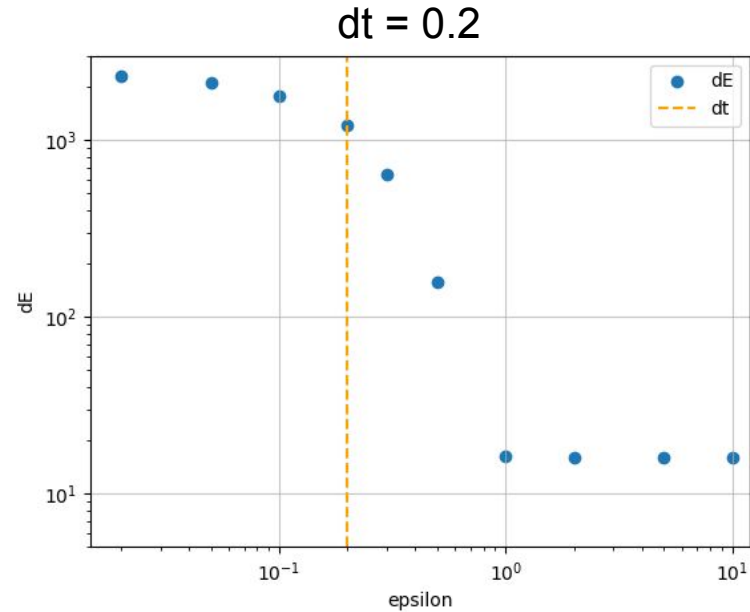
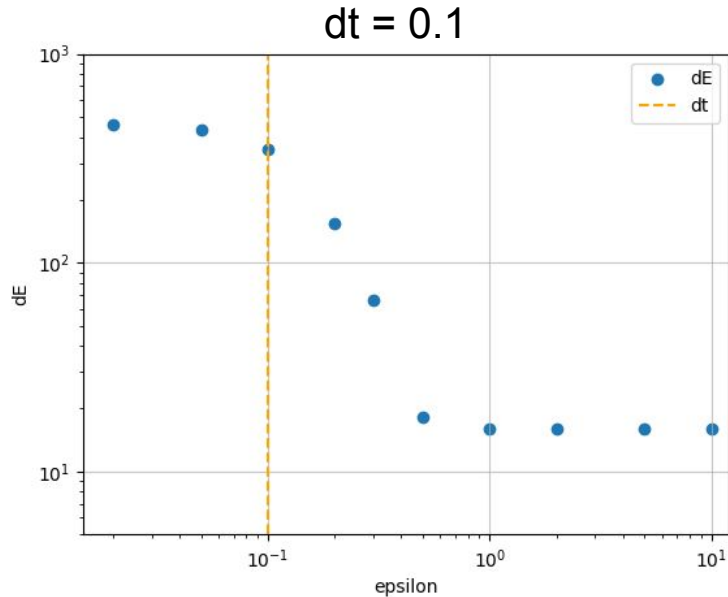
**Time cost of all methods
is ~ const**

**(x, v integration, a, K, F
computation)**

**Verlet just requires an
extra memory copy**

ε parameter has an effect on system energy and stability:

Verlet, $N = 1000$, iter = 1000



as a thumb rule, to ensure stability $\varepsilon \sim 5dt$

We identify the following parallelization opportunities:

1) the force loop itself

```
void compute_forces(int N, double epsilon, double x[][3], double m[], double F[][3], double *U_out) {  
    double U = 0.0, eps2 = epsilon*epsilon; // reset to zero and compute eps2  
    for (int i = 0; i < N; i++){  
        for (int k = 0; k < 3; k++){  
            F[i][k] = 0.0;  
        }  
    }  
    for (int i = 0; i < N; i++){  
        for (int j = 0; j < i; j++) {  
            double dx = x[i][0]-x[j][0], dy = x[i][1]-x[j][1], dz = x[i][2]-x[j][2]; // distance  
            double r2 = dx*dx + dy*dy + dz*dz + eps2; // distance squared (faster than pow)  
            double inv_r = 1.0 / sqrt(r2);  
            double mimj = m[i] * m[j];  
            double coef = mimj * inv_r*inv_r*inv_r;  
            F[i][0]-=coef*dx; F[j][0]+=coef*dx; // exploit symmetry  
            F[i][1]-=coef*dy; F[j][1]+=coef*dy;  
            F[i][2]-=coef*dz; F[j][2]+=coef*dz;  
            U -= mimj * inv_r;  
        }  
    }  
    *U_out = U;  
}
```

We identify the following parallelization opportunities:

2) initial position setup (more on later)

```
int it = 0;
while (it < N) {
    for (int j = 0; j < 3; j++)
        x[it][j] = (double)rand() / (double)RAND_MAX * L;

    // first particle is the star
    if (star == 1 && it == 0){
        for (int j = 0; j < 3; j++){
            x[0][j] = (float)L/2.0;
        }
    }
    ensurement = true;
    for (int j = 0; j < it; j++) {
        double dx = x[it][0]-x[j][0], dy = x[it][1]-x[j][1], dz = x[it][2]-x[j][2];
        double r2 = dx*dx + dy*dy + dz*dz; // distance squared
        if (r2 < min_dist*min_dist) {      // squared vs squared
            ensurement = false;
            break;
        }
    }
    if (ensurement){
        it++;
        printf("placed particle %d\n",it);
    }
}
```

We identify the following parallelization opportunities:

3) 4) 5) kinetic energy, acceleration and integration steps

```
// energy computation (t)
// kinetic
K = 0;
for(int i = 0; i < N; i++){
    double temp = 0;
    for(int k=0; k<3; k++){
        temp += v[i][k] * v[i][k];
    }
    K += m[i] * 0.5 * temp;
}
```

```
// acceleration = F / m
for (int i = 0; i < N; i++)
    for (int k = 0; k < 3; k++)
        a[i][k] = F[i][k] / m[i];
```

```
// integration (moving from t to t+1)
for (int i = 0; i < N; i++)
    for (int k = 0; k < 3; k++) {
        v[i][k] += a[i][k] * dt;
        x[i][k] += v[i][k] * dt;
    }
```


Directives



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

We can exploit Jetson's 4 cores CPU with OpenMP
first we try a simple **static** approach

```
static void compute_forces(int N, double epsilon, double x[][3], double m[], double F[][3], double *U_out)
{
    double U = 0.0, eps2 = epsilon*epsilon; // reset to zero and compute eps2

    // loop on i: rows of F (each thread computes over a particle)
    #pragma omp parallel for reduction(+:U) schedule(static) num_threads(NUM_THREADS)
    for (int i = 0; i < N; i++){
        for (int k = 0; k < 3; k++){
            F[i][k] = 0.0; // reset forces to 0
        }
        // full j to N: no symmetry exploitation
        for (int j = 0; j < N; j++) {
```

each thread computes F for a particle and any given other: **race conditions are avoided**, but we **lose symmetry exploitation**

Other loops that can be parallelized:

```
// parallelization over i
#pragma omp parallel for reduction(+:K) schedule(static)
for(int i = 0; i<N; i++){
    double temp = 0;
    for(int k=0;k<3;k++){
        temp += v[i][k] * v[i][k];
    }
    K += m[i] * 0.5 * temp;
}
```

```
// acceleration = F / m ++ integration t+1 ++ boundary check
// parallelization over i
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++)
    for (int k = 0; k < 3; k++) {
        a[i][k] = F[i][k] / m[i];
        v[i][k] += a[i][k] * dt;
        x[i][k] += v[i][k] * dt;
    }
```

these are called in just one
#pragma call: spare time

Depending on the number of threads, the workload is distributed across cores:

```
Model: NVIDIA Jetson Nano 2GB Developer Kit - Jetpack 4.6.4 [L4T 32.7.4]
 1 [|||||] 100.0% 1.5GHz 3 [|||||] 96.0% 1.5GHz
 2 [|||] 15.8% 1.5GHz 4 [|||] 13.4% 1.5GHz
Mem [|||||] 1.1G/1.9G FAN [|||||] 0.0% 0RPM
Swp [|||||] 2.0M/989M Jetson Clocks: [running]
Emc [|||||] 1.6GHz 0% NV Power[0]: MAXN
Iram [|||||] 0k/252k(lfb 252kB) Uptime: 0 days 1:4:29
GPU [|||||] 0.0% 921MHz
Dsk [|||||] 25.1G/58.4G

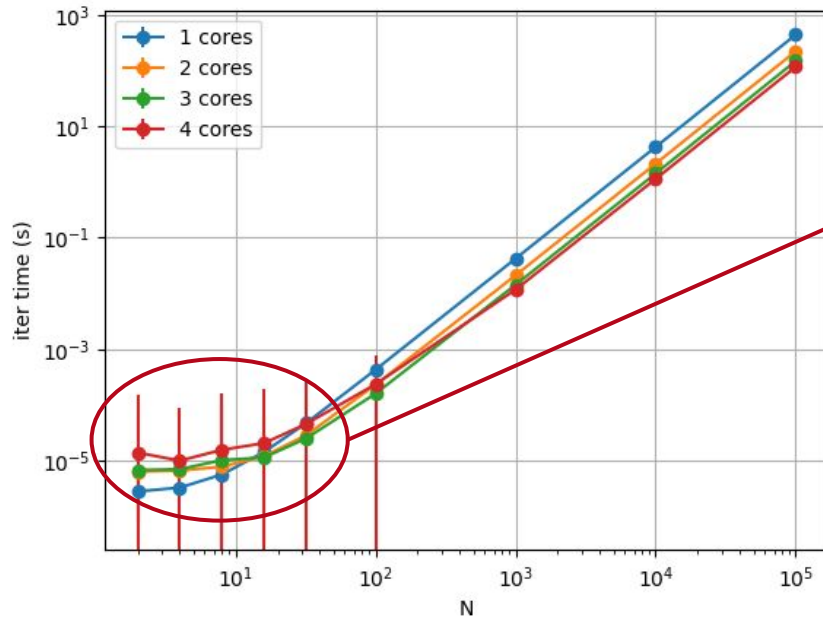
PID USER GPU TYPE PRI S CPU% MEM [GPU MEM] Command
4156 root I G 20 S 0.0 4.6M 76k nvargus-daemon
```

```
Model: NVIDIA Jetson Nano 2GB Developer Kit - Jetpack 4.6.4 [L4T 32.7.4]
 1 [|||||] 100.0% 1.5GHz 3 [|||||] 96.0% 1.5GHz
 2 [|||||] 96.0% 1.5GHz 4 [|||] 10.1% 1.5GHz
Mem [|||||] 1.2G/1.9G FAN [|||||] 0.0% 0RPM
Swp [|||||] 5.8M/989M Jetson Clocks: [running]
Emc [|||||] 1.6GHz 0% NV Power[0]: MAXN
Iram [|||||] 0k/252k(lfb 252kB) Uptime: 0 days 1:28:1
GPU [|||||] 0.0% 921MHz
Dsk [|||||] 25.1G/58.4G

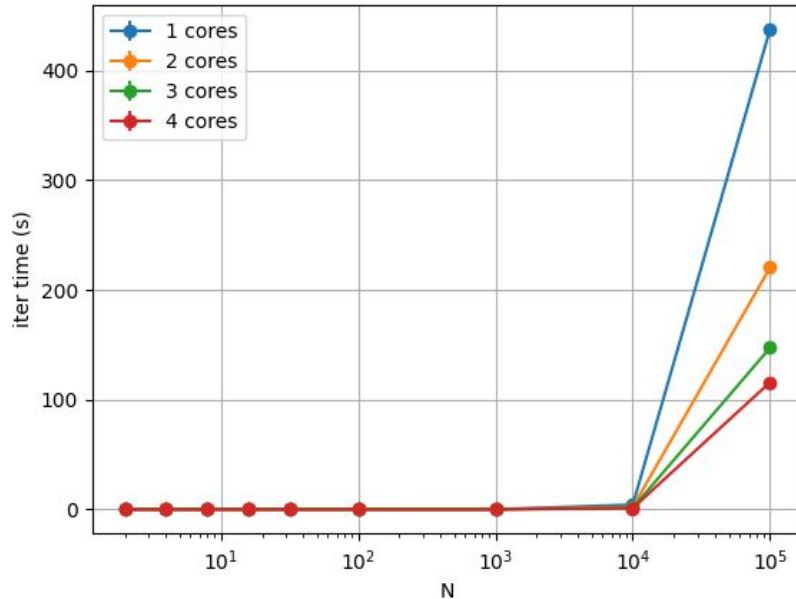
PID USER GPU TYPE PRI S CPU% MEM [GPU MEM] Command
4156 root I G 20 S 0.0 4.6M 76k nvargus-daemon
```

```
Model: NVIDIA Jetson Nano 2GB Developer Kit - Jetpack 4.6.4 [L4T 32.7.4]
 1 [|||||] 99.0% 1.5GHz 3 [|||||] 100.0% 1.5GHz
 2 [|||||] 99.0% 1.5GHz 4 [|||||] 100.0% 1.5GHz
Mem [|||||] 1.2G/1.9G FAN [|||||] 0.0% 0RPM
Swp [|||||] 5.8M/989M Jetson Clocks: [running]
Emc [|||||] 1.6GHz 0% NV Power[0]: MAXN
Iram [|||||] 0k/252k(lfb 252kB) Uptime: 0 days 1:26:34
GPU [|||||] 0.0% 921MHz
Dsk [|||||] 25.1G/58.4G

PID USER GPU TYPE PRI S CPU% MEM [GPU MEM] Command
4156 root I G 20 S 0.0 4.6M 76k nvargus-daemon
```



For low N latency is the dominant time factor: time does not change much and has **high variance**



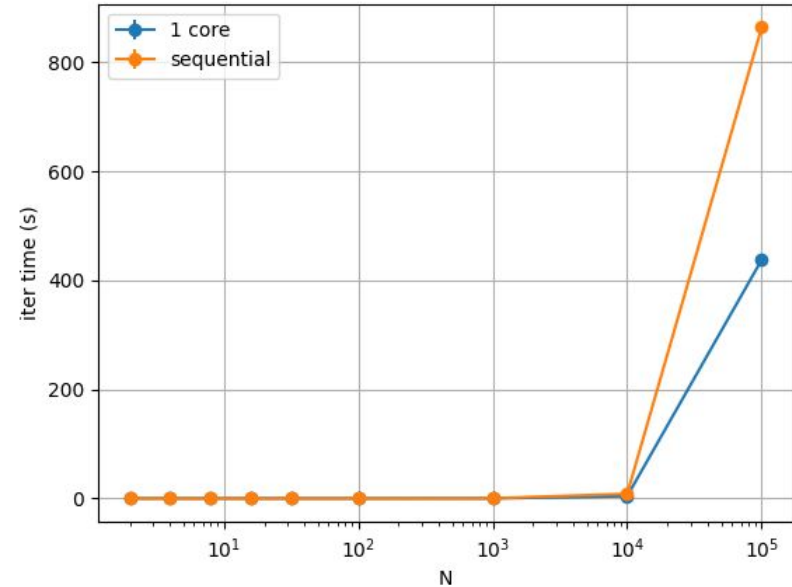
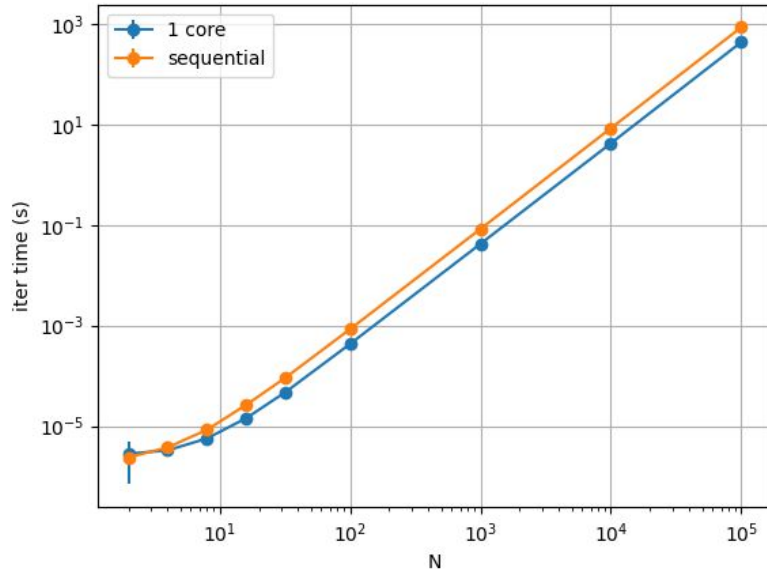
For low N latency is the dominant time factor: time does not change much and has **high variance**

As we increase N the trend becomes clear:
more cores are better

Excluding latencies, time goes as $1/n$ (n = available cores)

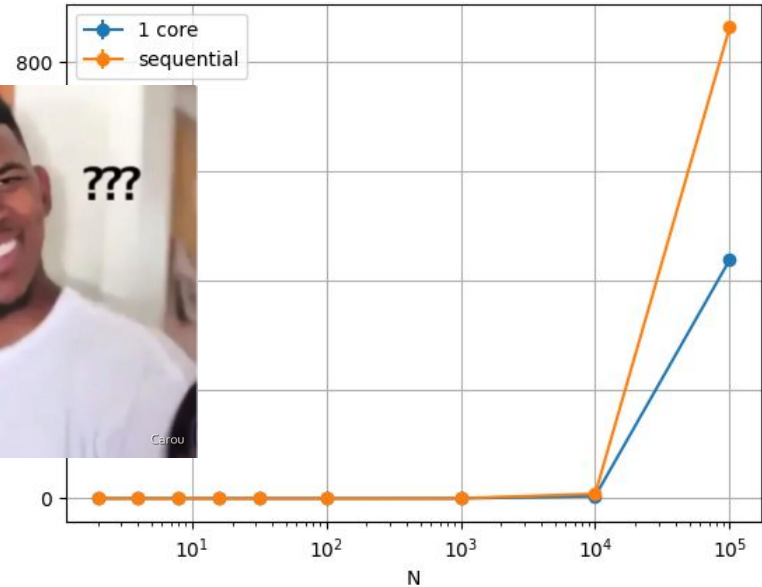
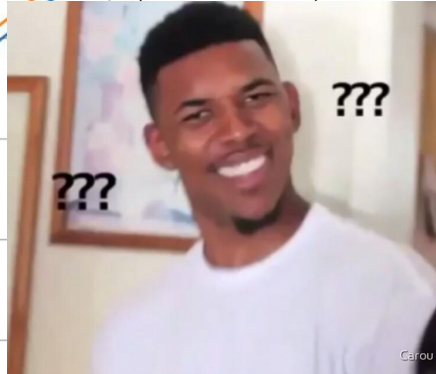
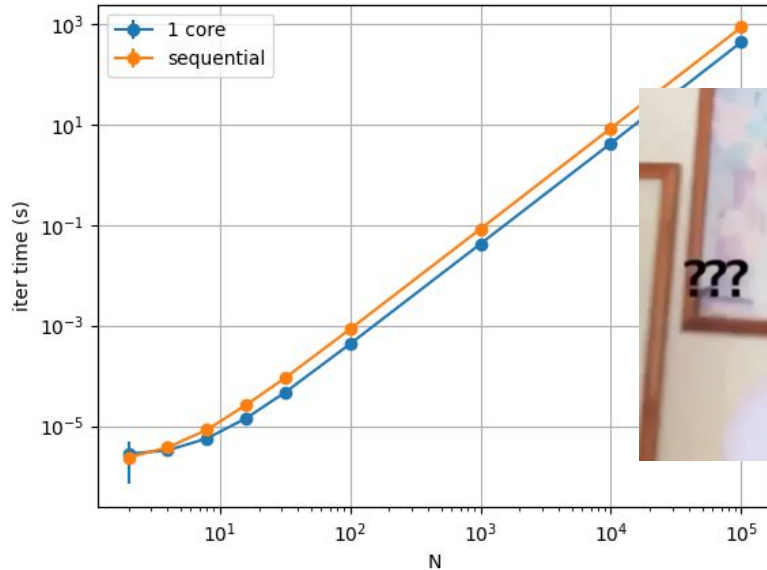
Sequential vs directives

Comparing sequential against 1 core I was surprised: how is 1 core beating sequential with **double the operations**?

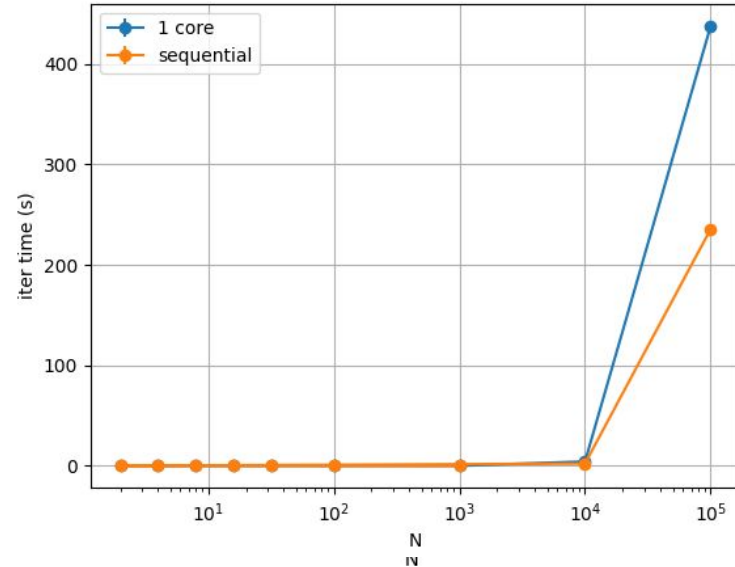
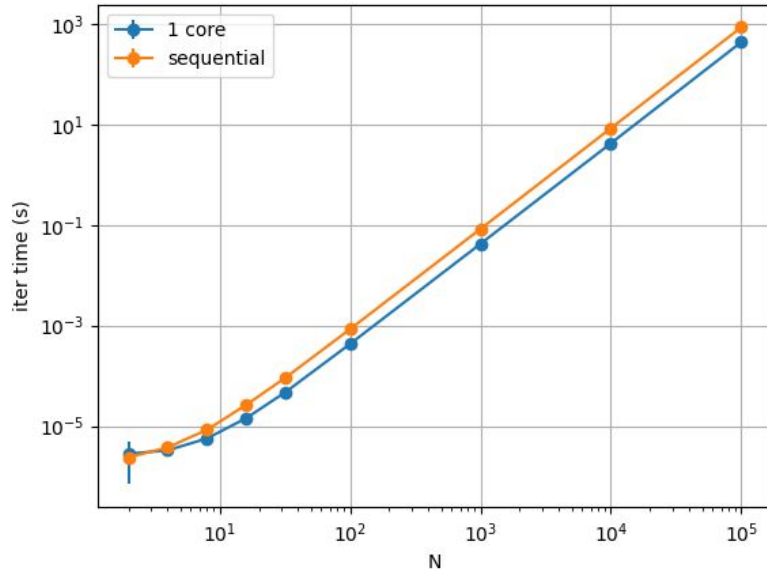


Sequential vs directives

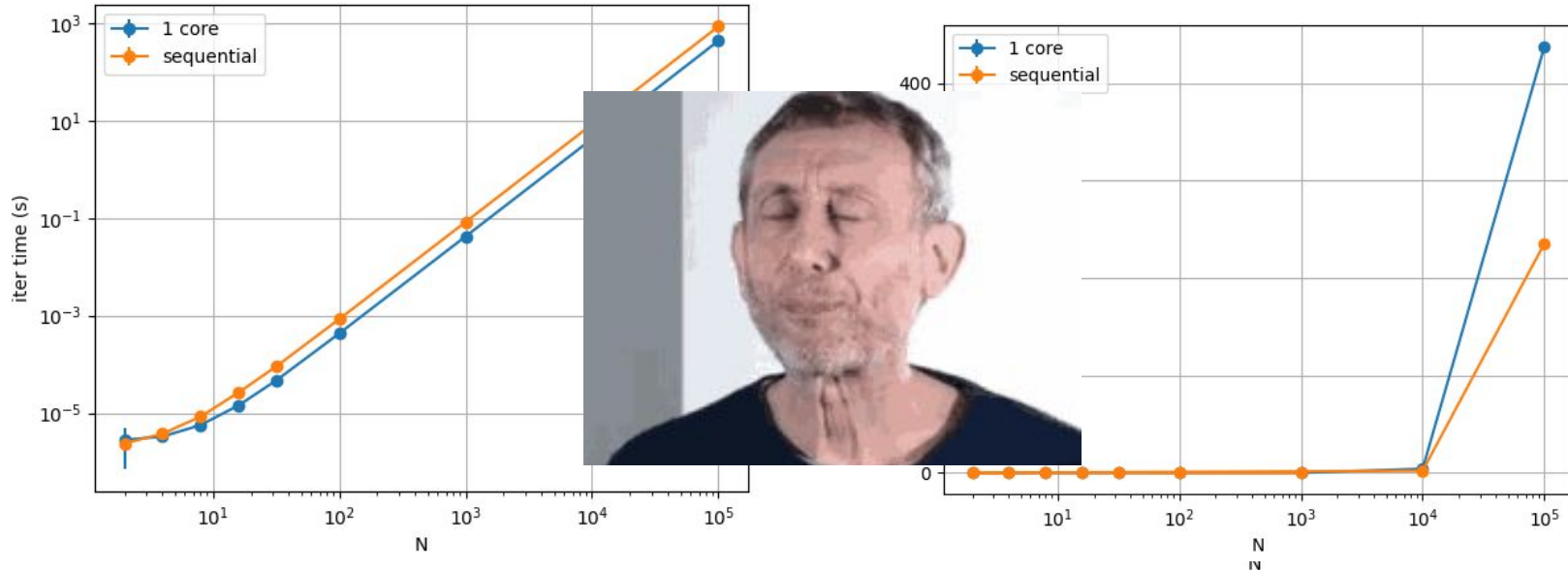
Comparing sequential against 1 core I was surprised: how is 1 core beating sequential with **double the operations**?



Turns out that **compiler choice matters**: by using the same compiler we observe a more reasonable trend:



Turns out that **compiler choice matters**: by using the same compiler we observe a more reasonable trend:



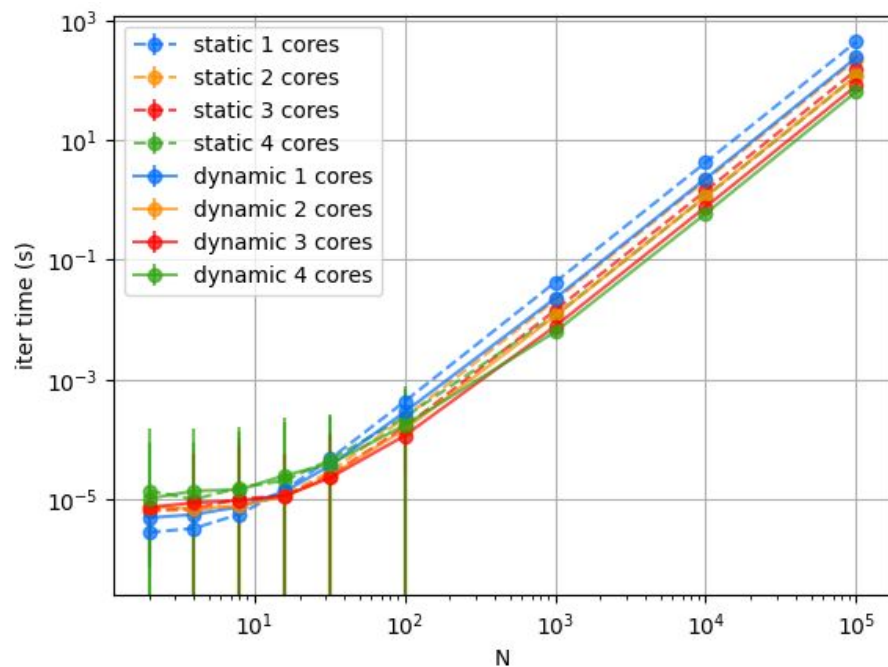
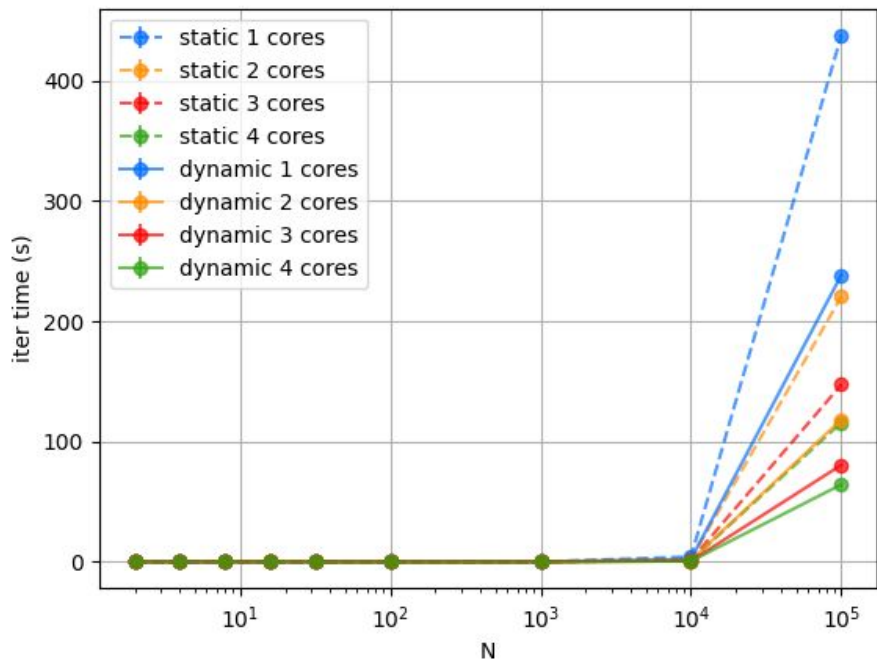
With static directives we lost the exploitation of **symmetry of forces**, leading to double the operations in force computation

We can schedule a **dynamic** approach to implement it back

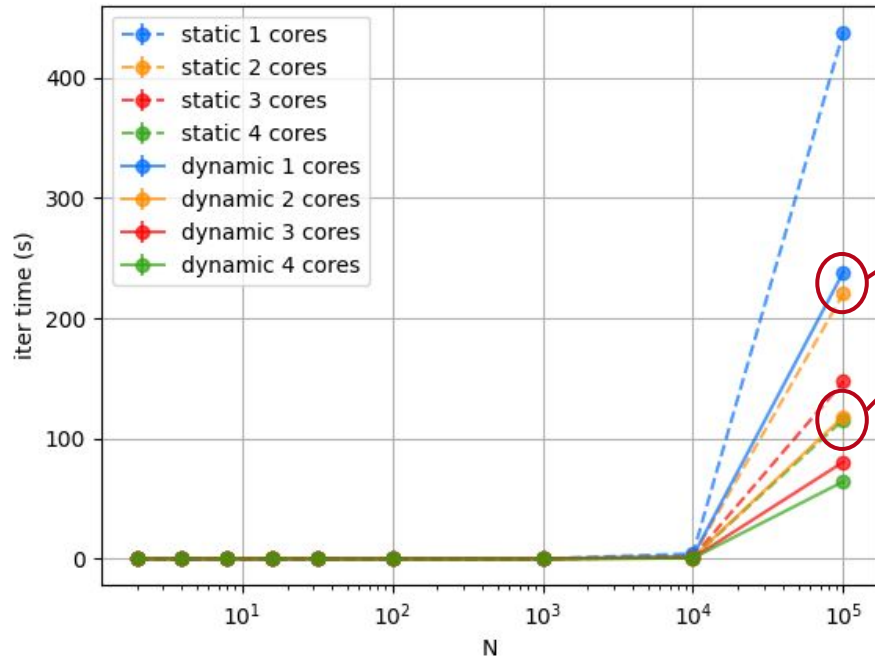
We exploit **symmetry** trading it for some latency time

```
#pragma omp for schedule(dynamic)
for (int i = 0; i < N; i++)
    for (int j = 0; j < i; j++) {
        double dx=x[i][0]-x[j][0], dy=x[i][1]-x[j][1], dz=x[i][2]-x[j][2];
        double r2=dx*dx+dy*dy+dz*dz+eps2, inv_r=1.0/sqrt(r2), mimj=m[i]*m[j];
        double coef=mimj*inv_r*inv_r*inv_r;
        Ft[i*3]-=coef*dx;
        Ft[i*3+1]-=coef*dy;
        Ft[i*3+2]-=coef*dz;
        // symmetry
        Ft[j*3]+=coef*dx;
        Ft[j*3+1]+=coef*dy;
        Ft[j*3+2]+=coef*dz;
        U -= mimj*inv_r;
    }
```

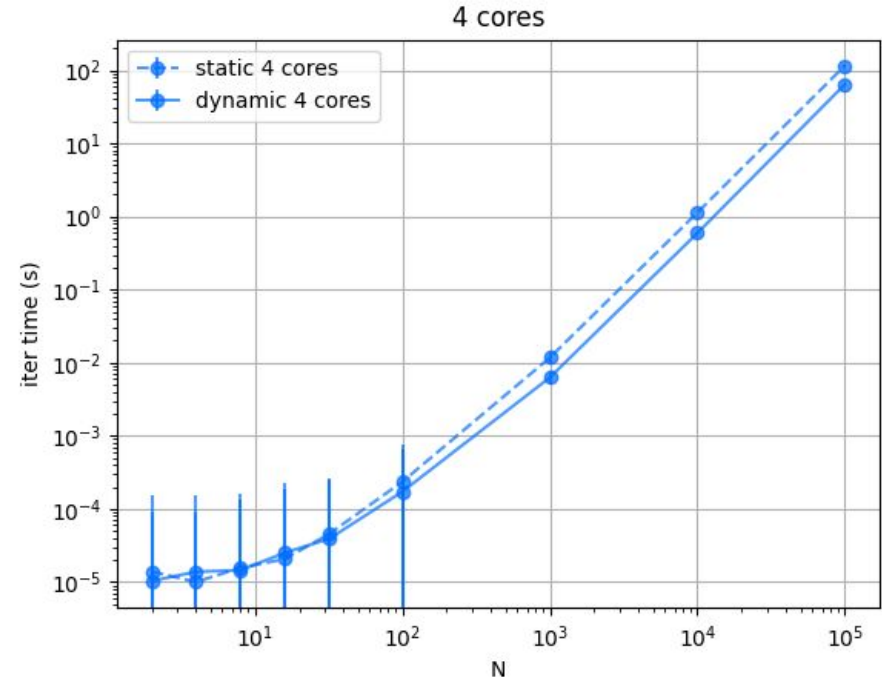
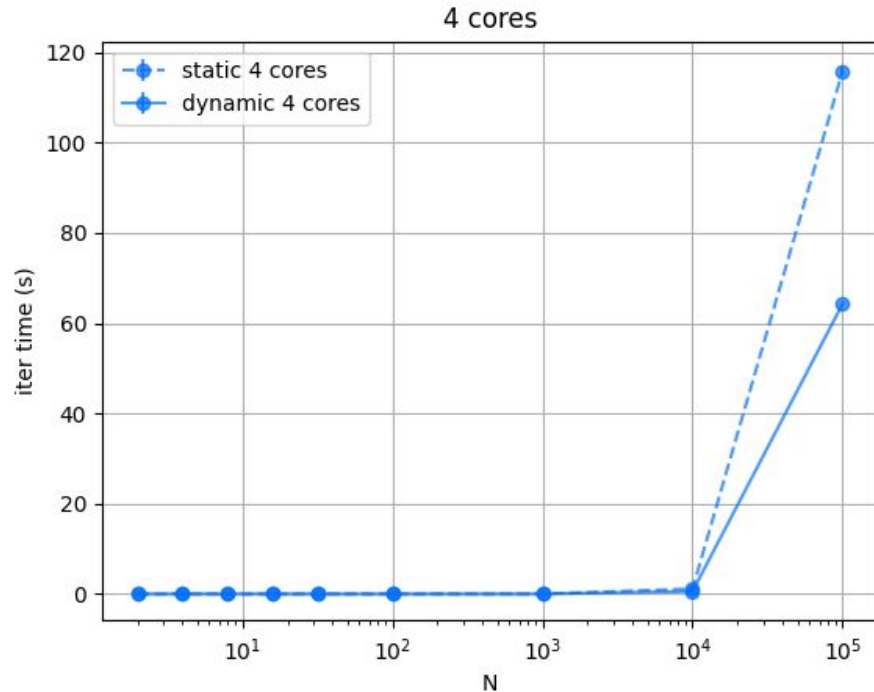
We observe a **performance improvement** with dynamic approach at **high N**



We observe a **performance improvement** with dynamic approach at high N



We observe a **performance improvement** with dynamic approach at high N



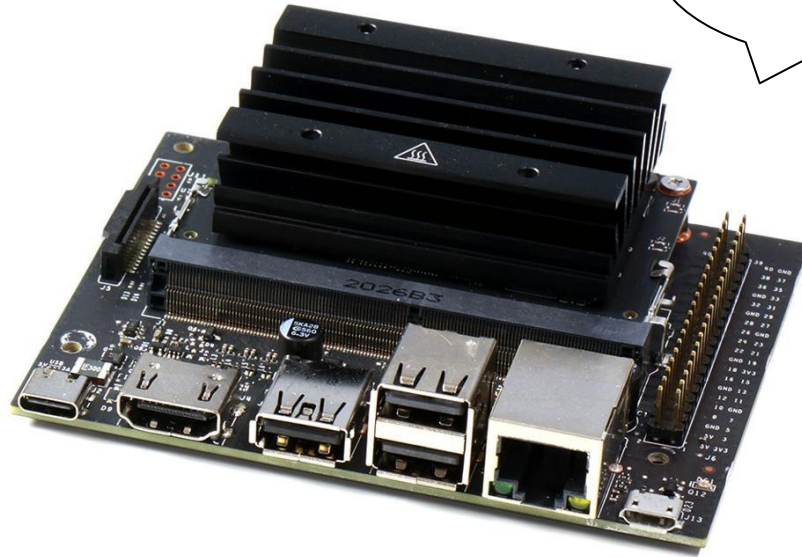
Parallel



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

This is what we're here for: the **parallel solution**

We'll be exploiting Jetson Nano's GPU
and features to provide an
optimized parallel
N-body solution

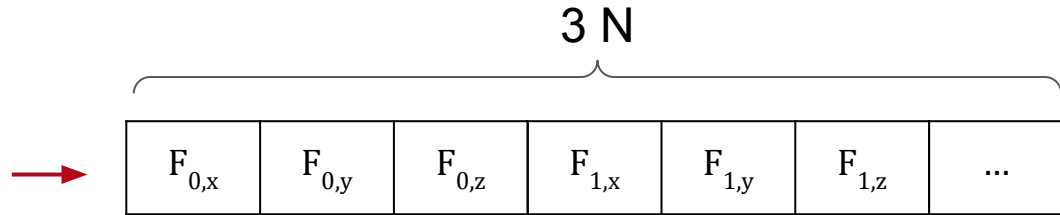


I can do...
MORE!

Our 3D vectors had a Nx3 matrix structure in previous codes:
the way we access the correct data is to define an index:

	3		
N	$F_{0,x}$	$F_{0,y}$	$F_{0,z}$
	$F_{1,x}$	$F_{1,y}$	$F_{1,z}$
	...	...	...
	$F_{N,x}$	$F_{N,y}$	$F_{N,z}$

```
#define IDX(i,k) ((i)*3+(k))
```



this way **i** index is deputed to pick a **particle**, and **k** index loops across **dimensions**

We define variables both in host (h_ prefix) and in device (d_ prefix)

```
float *h_x      = (float*)malloc(N*3*sizeof(float));  
float *h_v      = (float*)malloc(N*3*sizeof(float));  
float *h_a      = (float*)malloc(N*3*sizeof(float));  
float *h_a_old  = (float*)malloc(N*3*sizeof(float));  
float *h_F      = (float*)malloc(N*3*sizeof(float));  
float *h_m      = (float*)malloc(N*sizeof(float));
```

```
// device memory  
float *d_x, *d_v, *d_a, *d_a_old, *d_F, *d_m, *d_U, *d_K;  
CUDA_CHECK(cudaMalloc(&d_x,      N*3*sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_v,      N*3*sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_a,      N*3*sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_a_old, N*3*sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_F,      N*3*sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_m,      N*sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_U,      sizeof(float)));  
CUDA_CHECK(cudaMalloc(&d_K,      sizeof(float)));
```

Initialization is performed on CPU!
(more on this later)

CUDA abstraction:

```
const int    THREADS    = THREADS_PER_BLOCK;  
const int    BLOCKS     = (N + THREADS - 1) / THREADS;
```

CUDA copy
CPU to GPU:

```
CUDA_CHECK(cudaMemcpy(d_U, &zero, sizeof(float), cudaMemcpyHostToDevice));  
CUDA_CHECK(cudaMemcpy(d_K, &zero, sizeof(float), cudaMemcpyHostToDevice));
```

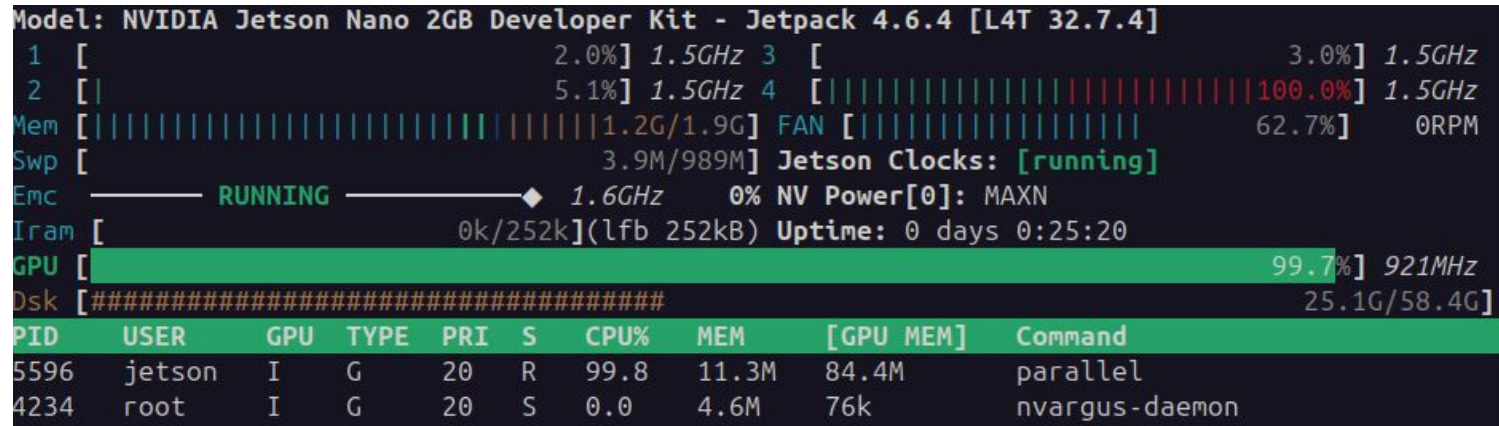
Kernel launches
and CUDA time
measurement:

```
cudaEventRecord(kernel_start);  
kernel_forces<<<BLOCKS,THREADS>>>(N, epsilon, d_x, d_m, d_F, d_U);  
CUDA_CHECK(cudaGetLastError());  
cudaEventRecord(kernel_stop);  
cudaDeviceSynchronize();  
cudaEventSynchronize(kernel_stop);  
cudaEventElapsedTime(&kms, kernel_start, kernel_stop);  
cudaEventDestroy(kernel_start);  
cudaEventDestroy(kernel_stop);  
  
kernel_accel<<<BLOCKS, THREADS>>>(N, d_F, d_m, d_a);  
kernel_kinetic<<<BLOCKS,THREADS>>>(N, d_v, d_m, d_K);  
kernel_euler<<<BLOCKS, THREADS>>>(N, d_x, d_v, d_a, dt);
```

CUDA to CPU (implicit sync):

```
CUDA_CHECK(cudaMemcpy(&U, d_U, sizeof(float), cudaMemcpyDeviceToHost));
CUDA_CHECK(cudaMemcpy(&K, d_K, sizeof(float), cudaMemcpyDeviceToHost));
```

Here the workload is on GPU:



Kernels:

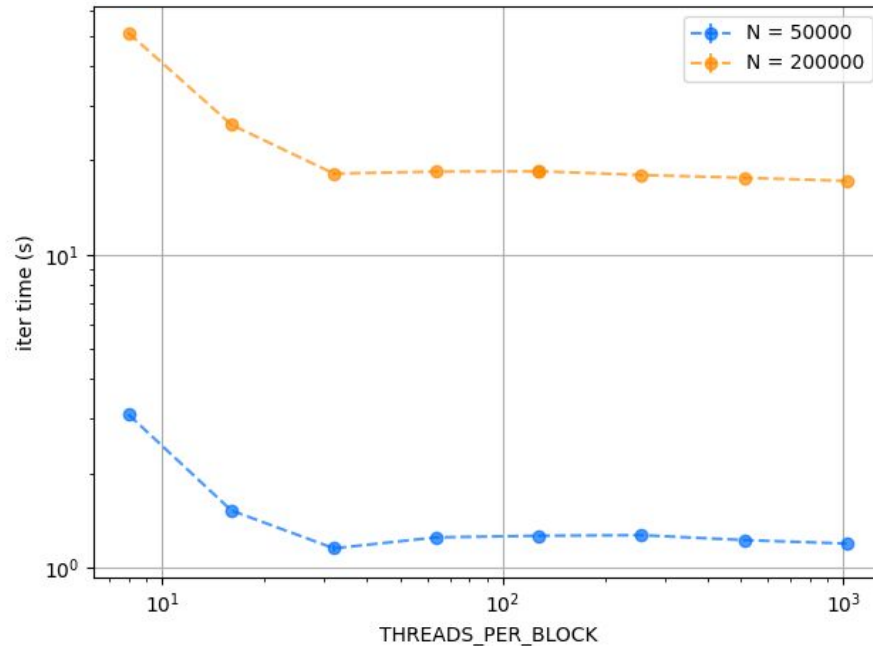
```
__global__ void kernel_forces(int N, float epsilon, float *x, float *m, float *F, float *U_out)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return; // memory bound check
    float Fi0 = 0, Fi1 = 0, Fi2 = 0, Ui = 0;
    float xi0 = x[IDX(i,0)]; // load once before j loop
    float xi1 = x[IDX(i,1)];
    float xi2 = x[IDX(i,2)];
    for (int j = 0; j < N; j++) {
        if (j == i) continue; // self force skip
        float dx = xi0 - x[IDX(j,0)]; // 1 FLOP
        float dy = xi1 - x[IDX(j,1)]; // 1 FLOP
        float dz = xi2 - x[IDX(j,2)]; // 1 FLOP
        float r2 = dx*dx + dy*dy + dz*dz + epsilon*epsilon;
        float inv_r = rsqrtf(r2); // 1 FLOP
        float inv_r3 = inv_r * inv_r * inv_r; // 3 FLOPs
        float mimj = m[i] * m[j]; // 1 FLOP
        Fi0 -= mimj * dx * inv_r3; // 3 FLOPs
        Fi1 -= mimj * dy * inv_r3; // 3 FLOPs
        Fi2 -= mimj * dz * inv_r3; // 3 FLOPs
        // do not count repeated
        if (j < i){
            Ui -= mimj * inv_r; // 2 FLOPs but half iter; consider as 1
        }
    }
    F[IDX(i,0)] = Fi0;
    F[IDX(i,1)] = Fi1;
    F[IDX(i,2)] = Fi2;
    // atomic add into global U
    atomicAdd(U_out, Ui);
}
```

Kernels:

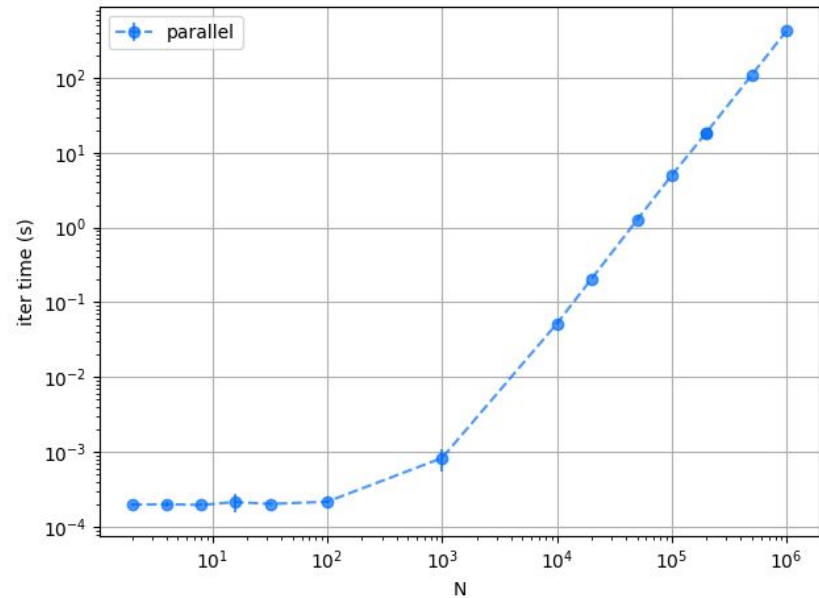
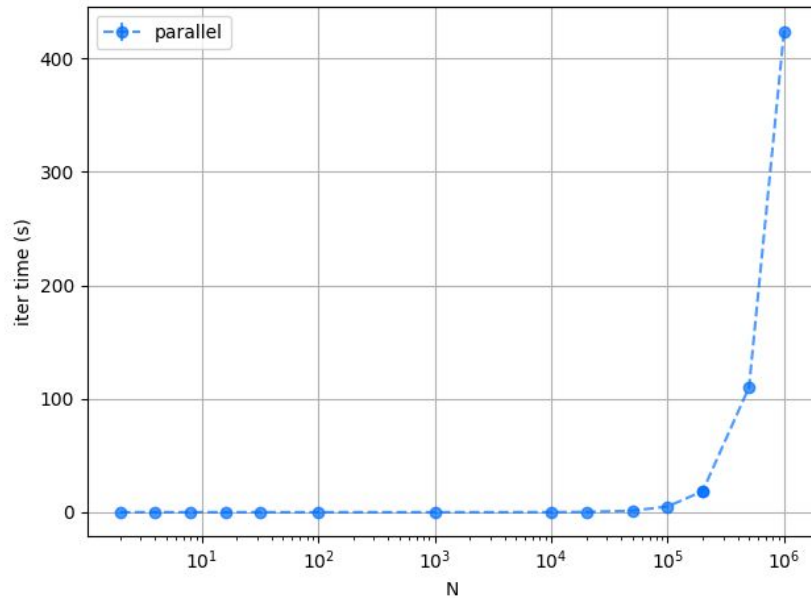
```
// a: loop across i
__global__ void kernel_accel(int N, float *F, float *m, float *a)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;
    for (int k = 0; k < 3; k++)
        a[IDX(i,k)] = F[IDX(i,k)] / m[i];
}
```

```
// K: accumulate onto i particle
__global__ void kernel_kinetic(int N, float *v, float *m, float *K_out)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;
    float v2 = v[IDX(i,0)]*v[IDX(i,0)] + v[IDX(i,1)]*v[IDX(i,1)] + v[IDX(i,2)]*v[IDX(i,2)];
    atomicAdd(K_out, 0.5 * m[i] * v2);
}
```

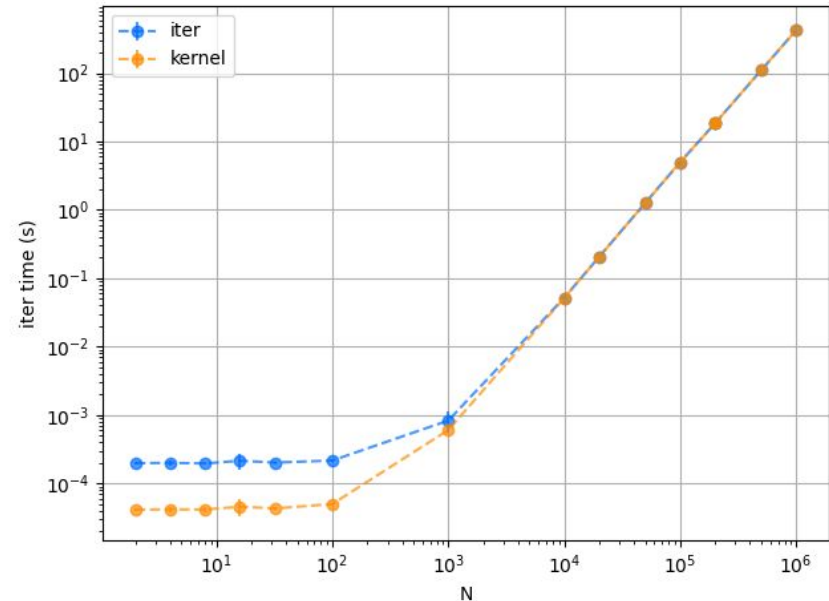
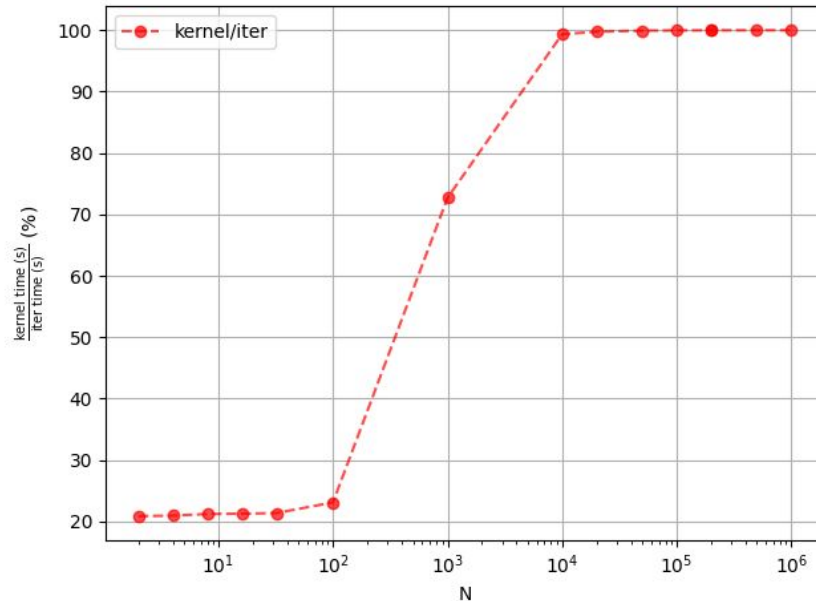

To explore THREADS_PER_BLOCK influence on the performance, we tried several configurations with different N (maximum for Jetson Nano is 1024):



We fix `THREADS_PER_BLOCK = 128` from now on. We now check how it goes as we change N :



Until now we have focused on iteration time. However, as we increase N , time required by **F kernel** becomes the most relevant piece:



Final Benchmark (iter time-euler)

			static				dynamic				
N	unit	seq	1 c	2 c	3 c	4 c	1 c	2 c	3 c	4 c	par
2	us	1.87	2.80	6.35	6.76	13.9	4.91	7.59	7.37	10.3	197.9
4	us	2.28	3.28	6.64	7.01	9.99	5.57	6.97	8.75	13.8	200.0
8	us	3.68	5.64	7.76	10.2	15.6	7.47	7.72	9.50	14.5	196.2
16	us	8.60	14.3	12.0	11.4	20.6	13.6	11.3	11.4	25.1	214.9
32	us	26.7	47.2	28.7	25.1	45.6	36.7	24.4	22.9	39.0	202.2
100	ms	0.226	0.429	0.230	0.159	0.235	0.286	0.147	0.115	0.172	0.217
1000	ms	22.3	42.1	21.1	14.3	11.7	22.7	11.5	7.76	6.26	0.830
10000	s	2.23	4.24	2.12	1.42	1.13	2.27	1.14	0.76	0.59	0.052
100000	s	235.7	437.8	220.5	147.3	115.9	238.4	118.3	80.6	64.3	4.97

Final Benchmark (iter time-euler)

			static				dynamic				
N	unit	seq	1 c	2 c	3 c	4 c	1 c	2 c	3 c	4 c	par
2	us	1.87	2.80	6.35	6.76	13.9	4.91	7.59	7.37	10.3	197.9
4	us	2.28	3.28	6.64	7.01	9.99	5.57	6.97	8.75	13.8	200.0
8	us	3.68	5.64	7.76	10.2	15.6	7.47	7.72	9.50	14.5	196.2
16	us	8.60	14.3	12.0	11.4	20.6	13.6	11.3	11.4	25.1	214.9
32	us	26.7	47.2	28.7	25.1	45.6	36.7	24.4	22.9	39.0	202.2
100	ms	0.226	0.429	0.230	0.159	0.235	0.286	0.147	0.115	0.172	0.217
1000	ms	22.3	42.1	21.1	14.3	11.7	22.7	11.5	7.76	6.26	0.830
10000	s	2.23	4.24	2.12	1.42	1.13	2.27	1.14	0.76	0.59	0.052
100000	s	235.7	437.8	220.5	147.3	115.9	238.4	118.3	80.6	64.3	4.97

Going crazy (extras)



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Thanks to parallelization it is possible to simulate a $N > 10^5$ system and finish before retirement

However our proposed approaches still need to face some issues:

- 1) Jetson watchdog and memory
- 2) Slow initialization
- 3) Very long times for a single iteration

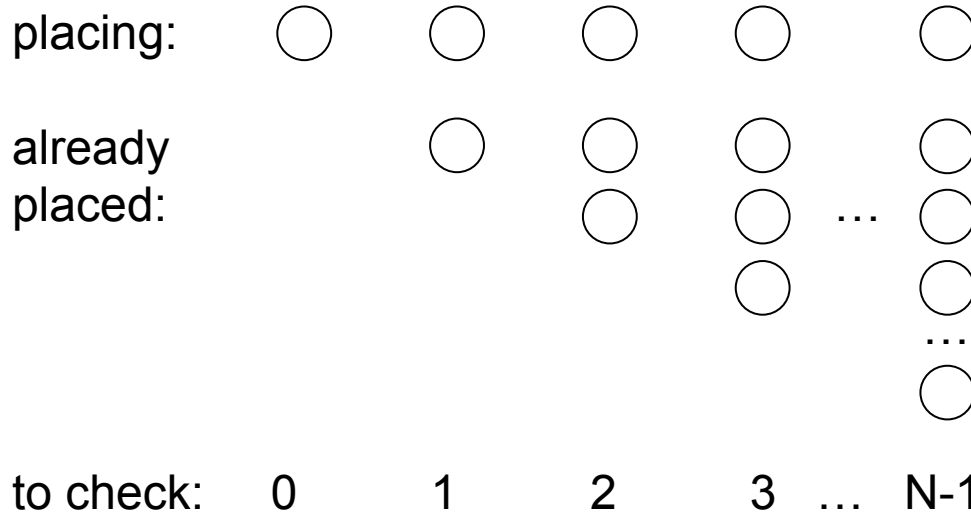
Let's take a look at 2) however!

How I'll look as Nbody finishes:



We are asked to initialize particles at a **minimum distance**

This task is **inherently sequential**, since it involves ensuring min distance with previously placed particles



This scales as N^2

So, why haven't we parallelized it yet?

Because it is not possible to do so efficiently or without significantly changing the task

2 approaches came to my mind:

- 1) keep particle placing sequential, parallelize distance check
- 2) subdivide space in subspaces and assign a subspace to each thread to fill respecting distance constraints

approach 1) involves launching *at best* $N - 1$ kernels (as N becomes larger it becomes hard to place particles) and memory copies

approach 2) cannot ensure both distance constraints **across different subspaces** and random uniform distribution in the box

idea: remove min dist (we have $\epsilon > 0$ to avoid singularities) and place randomly

We assume to be placing N particles in a box of size $L = 1000$, the density is: $\rho = \frac{N}{L^3}$

Considering the highest N of this work as $N = 10^6$ $\rho = \frac{N}{L^3} = 10^{-3}$

The expected average distance between nearest neighbors is provided treating placement as a poisson process:

$$\langle r_{min} \rangle = \frac{\Gamma(\frac{4}{3})}{\rho^{\frac{1}{3}}}$$

with the above density this becomes $\langle r_{min} \rangle = 5.54$
and deviation $\sigma = 1.92$

which implies that (at least at initialization) particles are **on average** far enough to avoid force imbalances

In our previous parallel implementation **all threads accessed data from slow shared memory**

An idea to reduce latency impact is to divide data in **tiles** and have each thread access its own **block memory**

this enables the use of **fast SM of blocks**

```
// tile memory
__shared__ float s_x[THREADS_PER_BLOCK * 3];
__shared__ float s_m[THREADS_PER_BLOCK];
```

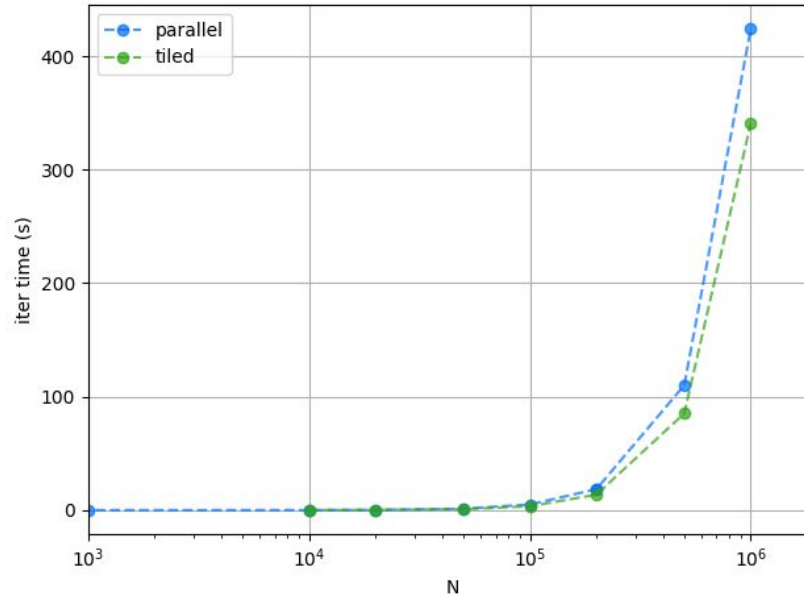
cost:
one additional
__syncthreads();
call!

```
// tile subdivision
for (int tile = 0; tile < (N + blockDim.x - 1) / blockDim.x; tile++) {
    int j_load = tile * blockDim.x + threadIdx.x;
    if (j_load < N) {
        s_x[threadIdx.x*3+0] = x[IDX(j_load,0)];
        s_x[threadIdx.x*3+1] = x[IDX(j_load,1)];
        s_x[threadIdx.x*3+2] = x[IDX(j_load,2)];
        s_m[threadIdx.x]      = m[j_load];
    }
    __syncthreads(); // ensure memory sync before moving on
}
```

Parallel solution (tiled)

58

Here we show the comparison between **tiled** and basic **parallel**



We have previously treated GPU and CPU memories as **separated**, as in standard structures

However, Jetson Nano has **shared** CPU and GPU memory: we can **exploit** this by reducing memory related latencies!



We have previously treated GPU and CPU memories as **separated**, as in standard structures

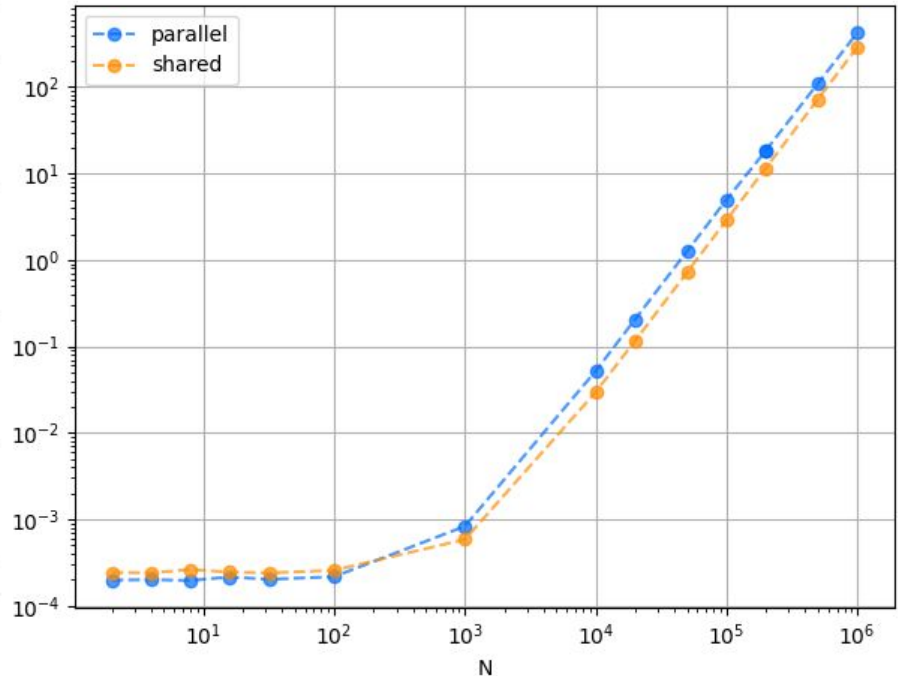
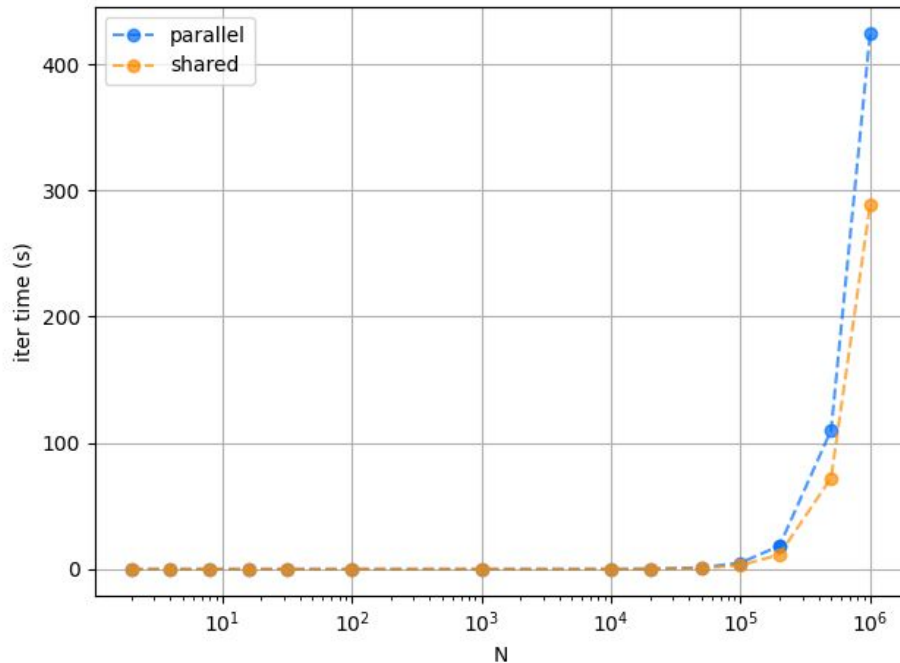
However, Jetson Nano has **shared** CPU and GPU memory: we can **exploit** this by reducing memory related latencies!



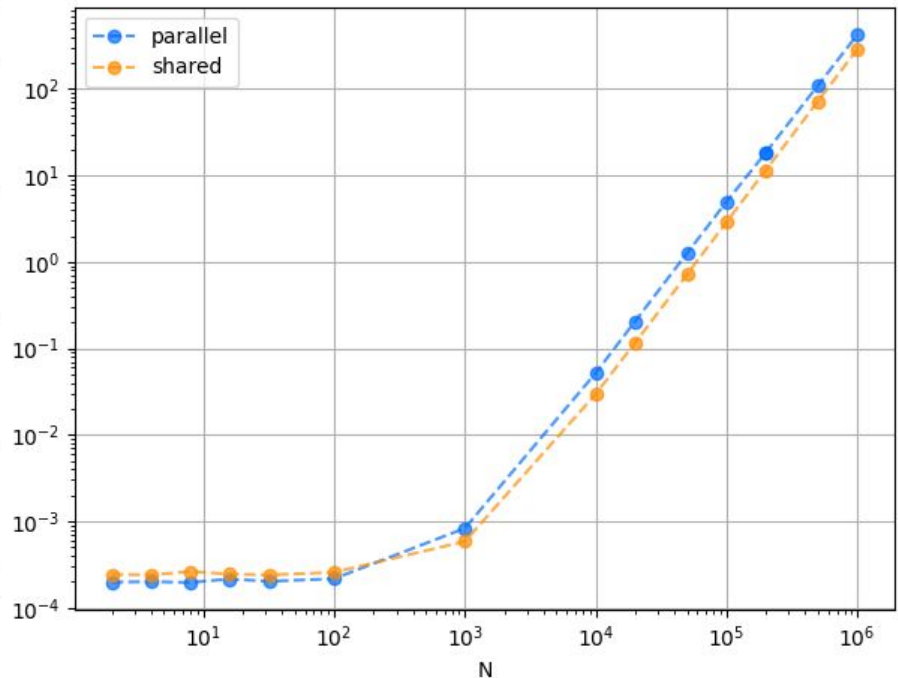
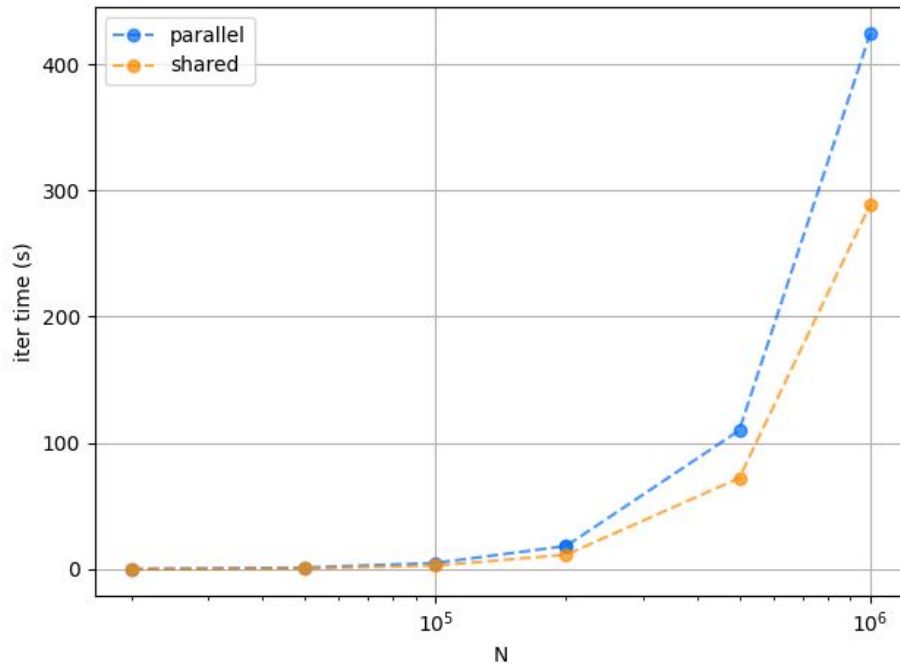
Main changes of code involve variable allocation (and some more syncthread calls): we just now need only one shared instance of each variable and ensure no race condition is met

```
// variables allocation (SHARED)  
float *x, *v, *a, *old_a, *m, *F, *U, *maxa2, *mind, *K;  
CUDA_CHECK(cudaMallocManaged(&x,      bytes_x));  
CUDA_CHECK(cudaMallocManaged(&v,      bytes_v));  
CUDA_CHECK(cudaMallocManaged(&a,      bytes_a));  
CUDA_CHECK(cudaMallocManaged(&old_a, bytes_a));  
CUDA_CHECK(cudaMallocManaged(&m,      bytes_m));  
CUDA_CHECK(cudaMallocManaged(&F,      bytes_F));  
CUDA_CHECK(cudaMallocManaged(&U,      bytes_U));  
CUDA_CHECK(cudaMallocManaged(&maxa2, sizeof(float)));  
CUDA_CHECK(cudaMallocManaged(&mind,  sizeof(float)));  
CUDA_CHECK(cudaMallocManaged(&K,      sizeof(float)));
```

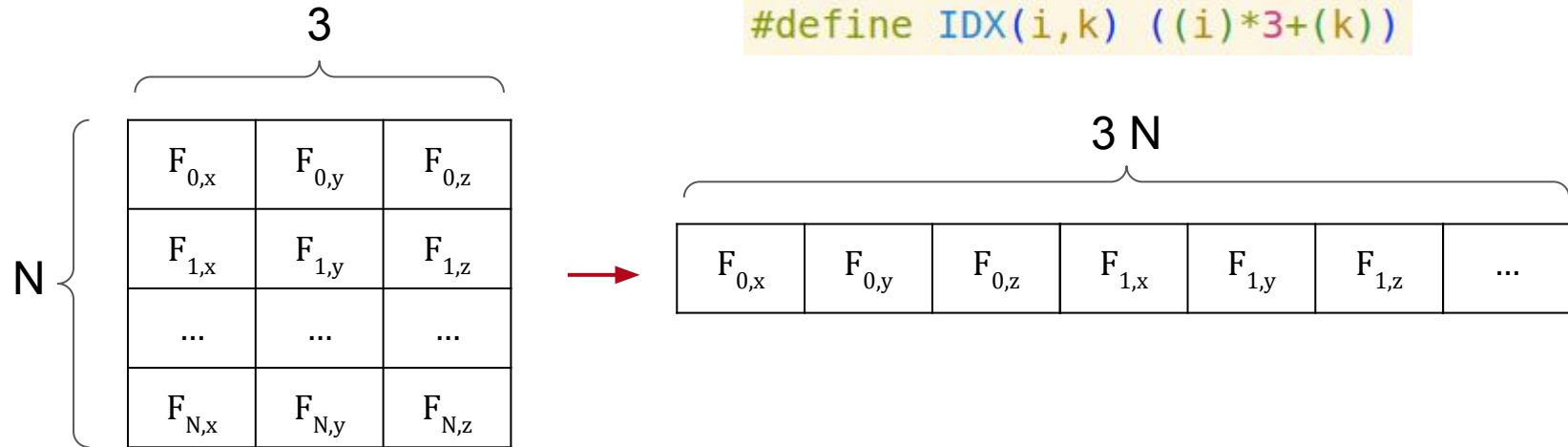
Here we have a comparison between shared memory and separated memory with the same simulation settings (euler, TPB = 128, dt = 0.01, epsilon = 0.2):



Here we have a comparison between shared memory and separated memory with the same simulation settings (euler, TPB = 128, dt = 0.01, epsilon = 0.2):



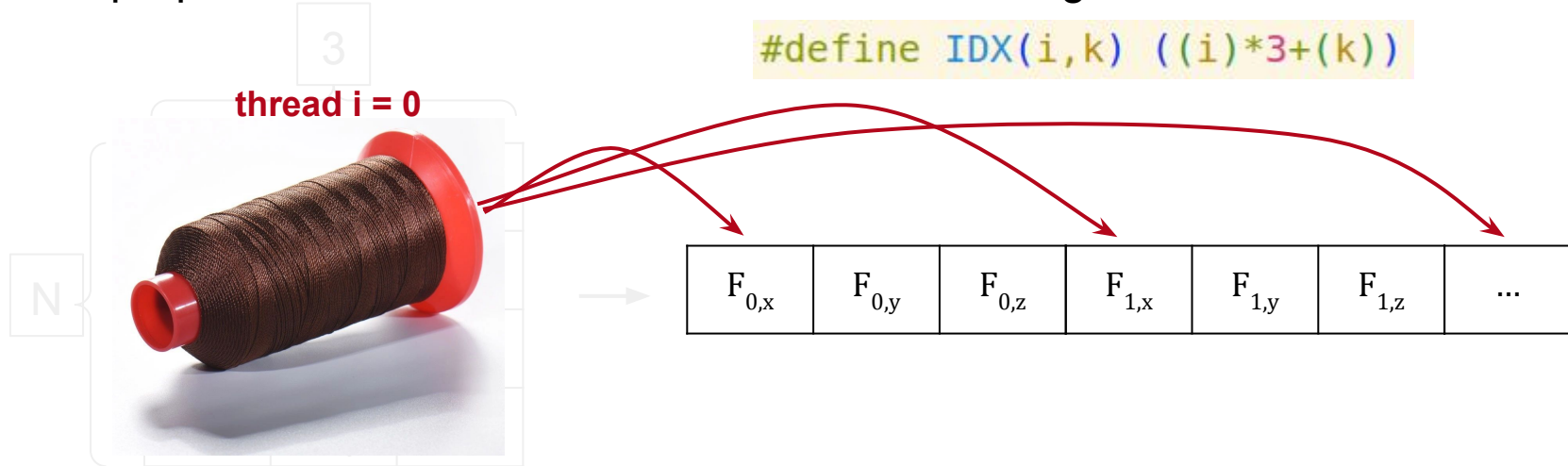
The proposed data structure until now was a 1D tiling with an index:



```
#define IDX(i,k) ((i)*3+(k))
```

while very easy to understand, memory access is **inefficient!**

The proposed data structure until now was a 1D tiling with an index:



while very easy to understand, memory access is **inefficient!**

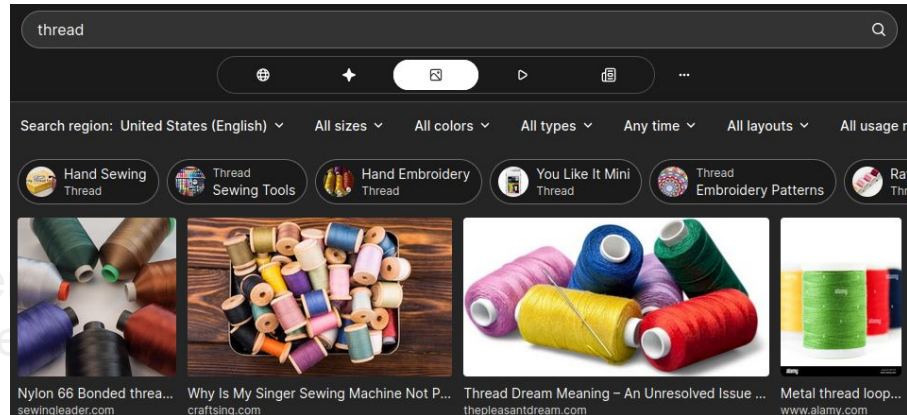
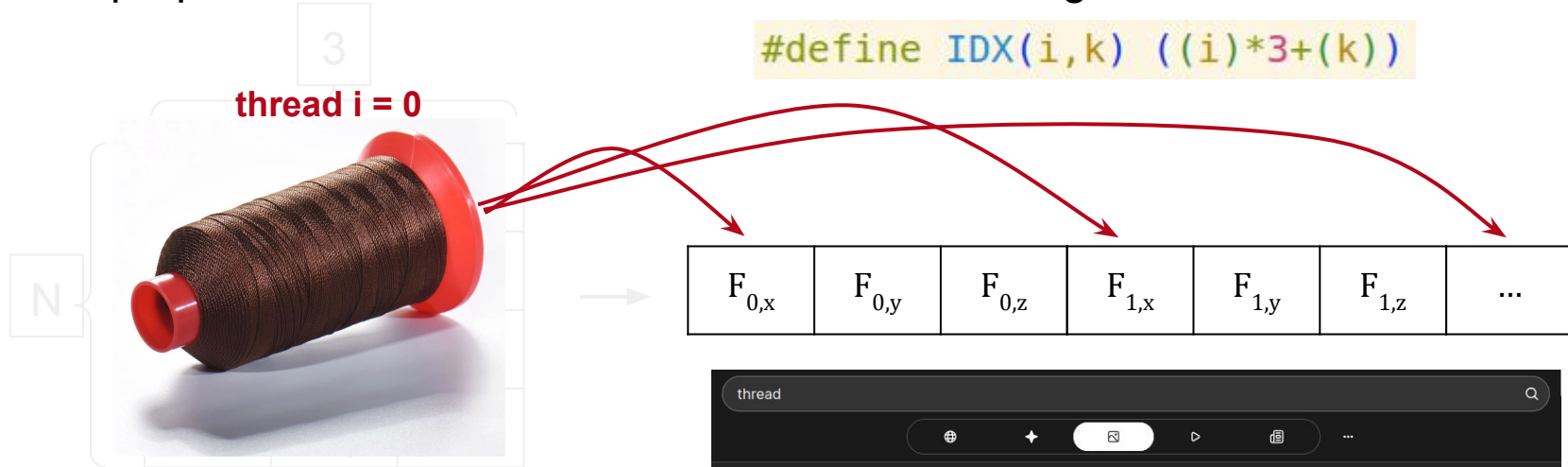
Threads can spend less time in memory access if they need to read **close** locations!

Parallel solution (coalesced)

66

The proposed data structure until now was a 1D tiling with an index:

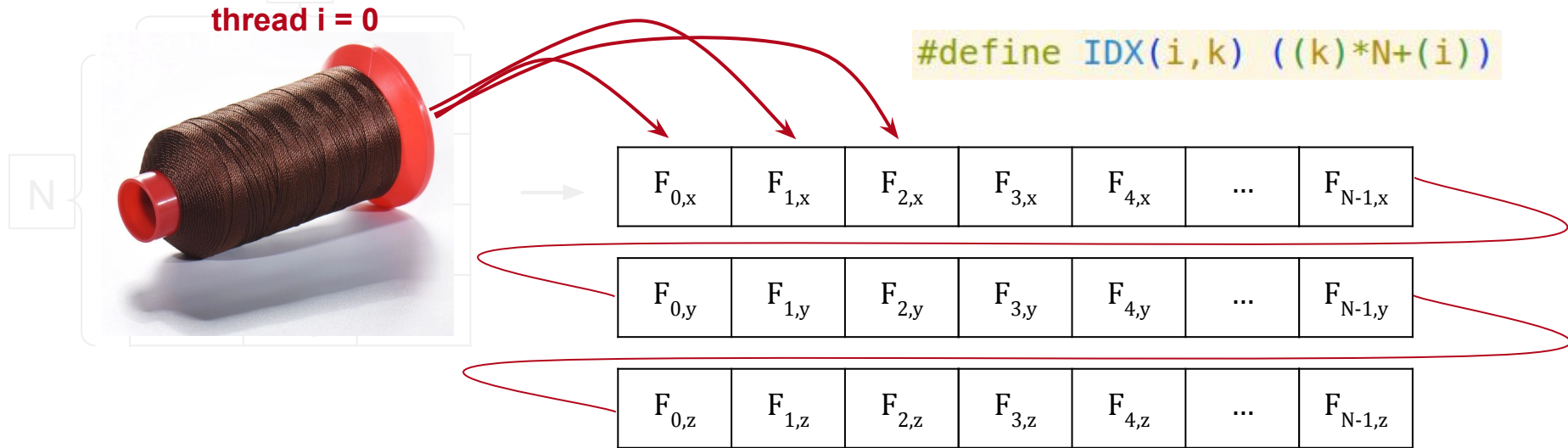
```
#define IDX(i,k) ((i)*3+(k))
```



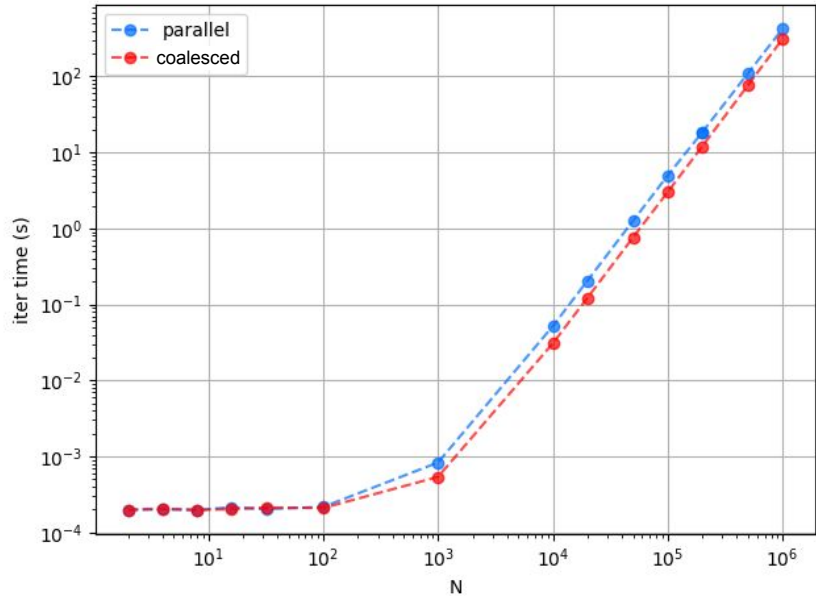
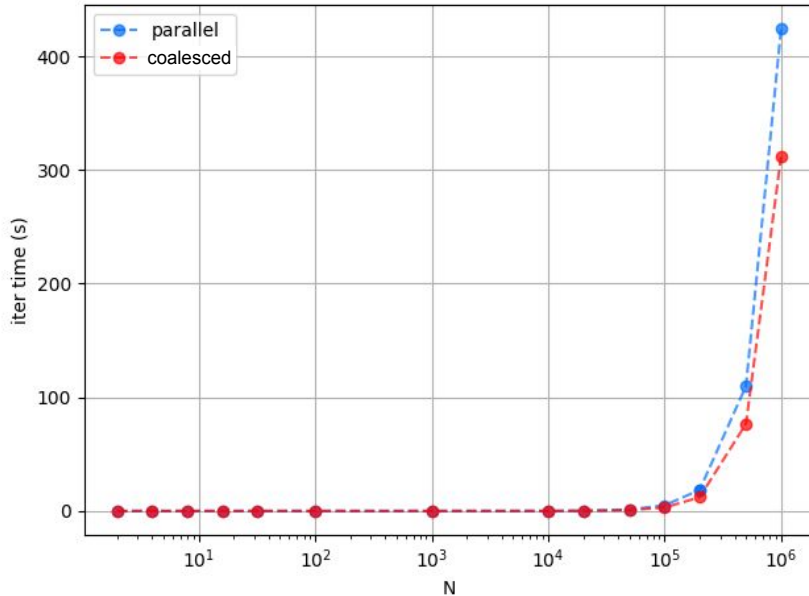
while very easy to understand, me
Threads can spend less time in me

cations!

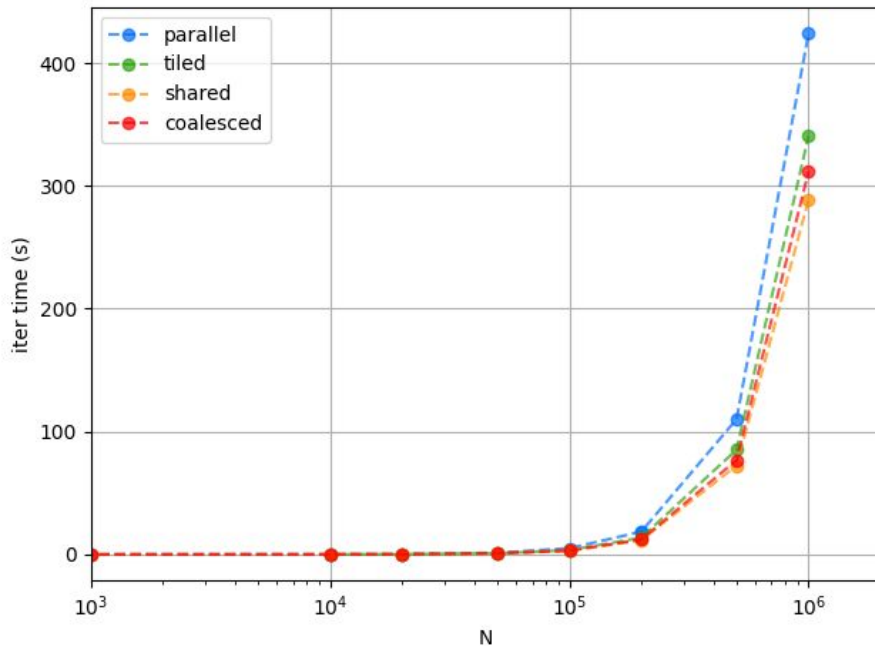
Idea: define a **vector** aligning all coordinates one by one : this way each thread accesses adjacent memory indices for computing force of particle i



The code follows the same logic as **standard parallel** with a **different index**
We then proceed to simulate and compare iter times (euler, TPB = 128, dt = 0.01, epsilon = 0.2):

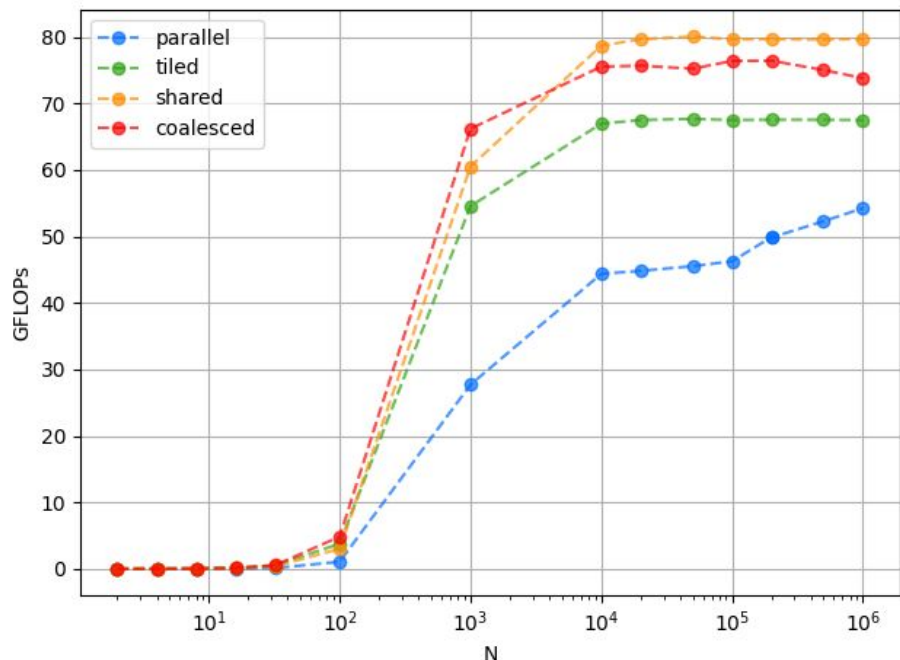


For very high N **shared memory** approach **wins!** (times are in s)



N	parallel	tiled	shared	coalesced
10^4	0.052	0.035	0.030	0.031
$2 \cdot 10^4$	0.205	0.137	0.116	0.122
$5 \cdot 10^4$	1.26	0.851	0.729	0.765
10^5	4.97	3.40	2.89	3.01
$2 \cdot 10^5$	18.4	13.6	11.6	12.0
$5 \cdot 10^5$	109.9	85.4	72.2	76.7
10^6	424.1	340.7	288.6	311.7

We also checked GPU usage of force kernels. It involves **23 FLOPs** and a series of memory accesses



In the plot we report just the number of FLOPs relative to force kernel, since for high N it contributes for 99.99% of the workload

Measured:

SFU (rsqrt) : 29.2 GOps/s

FMA : 222.4 GFLOPs

Our best (F) ~ 80 GFLOPs



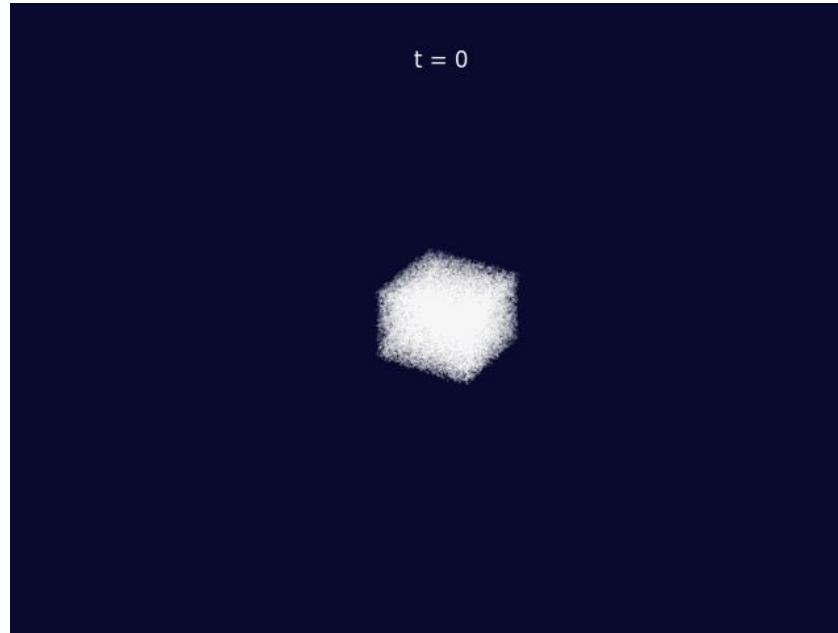
we're running at ~ 35% of FMA speed!

We are then enabled to simulate star clusters. We can even add velocity distributions such as **Maxwell-Boltzmann**

```
// MAXWELL-BOLTZMANN branch
if(mb == 1){
    for(int i = 0; i < N; i++){
        double sigma = sqrt(T / h_m[i]); //  $v \sim N(0, kb*T/M)$ 
        h_vx[i] = sigma * gaussian();
        h_vy[i] = sigma * gaussian();
        h_vz[i] = sigma * gaussian();
    }
}
```

```
if (star == 1){
    h_m[0] = 250;
    printf("star mode activated!\n");
}
```

We are then enabled to simulate star clusters. We can even add velocity distributions such as **Maxwell-Boltzmann**



We are then enabled to simulate star clusters. We can even add velocity distributions such as **Maxwell-Boltzmann** (verlet!)

